

LumiCity Kerala

Comprehensive System Documentation

A highly detailed architectural, algorithmic, and code-level deep dive into the smart street light intensity prediction system.

Includes: Dashboard UI walkthrough, complete algorithms, system block diagrams, and automatic data refresh architecture.

Part 1: System Introduction & Deep Architectural Foundation

1.1 Executive Architectural Summary & Vision

LumiCity Kerala is a paradigm-shifting, AI-driven Smart City Street Light Control Portal and production execution pipeline strictly tailored for the complex municipal geometries of Kerala. Its definitive, unwavering goal is to structurally dissect and solve the dual crisis of severe light pollution (skyglow, ecological disruption) and unsustainable municipal energy consumption.

Unlike heuristic, legacy, or purely timer-based mechanical systems that dictate infrastructure behavior based on arbitrary chronological clocks, LumiCity operates an aggressively continuous mathematical and data-driven approach. It predicts the `_Absolute Minimal Required Intensity_` for every individual street light fixture in a city grid in absolute real-time. It balances this minimization objective against strict safety constraints triggered by localized fog, intense commuter traffic, or specific civil celebrations.

1.2 The Physics and Ecology of Light Pollution

To understand the engineering architecture of LumiCity, we must first profoundly examine the physical problem domain it intends to optimize: Light Pollution.

The Typology of Unnatural Illumination

1. **Skyglow:** The brightening of the night sky over inhabited areas. When uncontrolled LED photons emitted from high-mast structural lighting project horizontally or upwards (due to poor baffling or excessive raw lumen output), they scatter off atmospheric moisture, aerosols, and macroscopic dust particles (Rayleigh and Mie scattering). This creates an omnipresent luminous dome over a city, eradicating the visibility of celestial bodies and massively disrupting nocturnal avian migration tracking.
2. **Light Trespass:** Hard, uncontrolled photon spillage falling where it is unintended or unneeded (e.g., streaming into residential windows, disrupting human melatonin synthesis sequences and circadian stability).
3. **Disability Glare:** An extreme contrast generated by an overly bright light source against a totally dark background. This actively harms the human visual system, causing pupillary constriction that `_reduces_` visibility in the shadows behind the light. A street light operating at 100% capacity at 3 AM does not increase safety; it creates absolute blackness in the unlit periphery, hiding hazards.

The Optimization Mandate

Classical street lighting is binary and blunt: OFF during the day, 100% ON during the night.

LumiCity posits a highly complex continuous optimization problem represented by the function:

``Intensity_req = f(Time, Radiance_VIIRS, Visibility, Traffic_Risk, Event_Multiplier, AmbientLux)``

The objective function specifically seeks to **minimize** `Intensity_req` (hard-bounded between an output amplitude of 0% and 100%) subject to strict, unyielding civic safety constraints-primarily that `Visibility_poor` implies `Intensity_req >= 60%`.

This mathematical formulation guarantees that the municipality inherently saves maximum kinetic energy (measured in raw kilowatts) during clear, low-traffic deep nights, while retaining the capacity to instantaneously surge intensity during visually or physically dangerous atmospheric conditions like a localized Monsoon deluge.

1.3 Core Technological Stack & Execution Flow

The platform avoids monolithic C# or Java backend bloat, instead favoring a robust, highly modular Python 3.10+ centric stack. This architecture isolates core computational concerns into distinct execution layers: Physical Measurement, Offline Machine Learning, Edge Inference Surveillance, and Asynchronous Presentation.

1.3.1 Frontend Presentation Layer: Streamlit

- **Why Streamlit?** Instead of wrestling with a decoupled React.js/Next.js Single Page Application (SPA) architecture requiring a massive FastAPI middleware layer for simple SQL hydration, Streamlit fuses rapid reactive prototyping with production-tier dashboarding mechanisms.
- **Functional Purpose:** It facilitates the uninterrupted creation of interactive geospatial structures (`folium``), handles real-time metric scalar updates, and allows civic engineers immediate structural Database manipulation (Manual Overrides) without SPA memory overhead or DOM rehydration lag.

1.3.2 Structural Persistence: SQLite3 (`kerala_smart_lights.db``)

- **Why SQLite?** While modern architectures default to PostgreSQL or MongoDB, LumiCity operates primarily as an isolated Edge appliance. A file-bounded relational database (`.db``) guarantees absolute localized control, eliminating network latency when the Streamlit UI requests rapid frame-by-frame baseline prediction recalculations from the ML model outputs.

1.3.3 Edge Computing Surveillance: YOLOv8 (Ultralytics)

- **Deep Learning on the Periphery:** YOLOv8 represents the absolute state-of-the-art in single-shot structural object detection tensors.
- **Implementation Strategy:** In this pipeline, YOLOv8 acts as an unblinking, totally localized edge sensor. It assesses raw pixel geometries to calculate a dynamic Traffic Risk Score (TRS). By threading this operation into an isolated background loop, it effectively bypasses Python's Global Interpreter Lock (GIL) limitations, ensuring the Streamlit UI matrix remains completely non-blocking and highly responsive.

1.3.4 Predictive Engine: `GradientBoostingRegressor``

- **Why not Deep Learning for Intensity?** Generative Adversarial Networks (GANs) or deep Multi-Layer Perceptrons (MLPs) are notoriously complex, require vast computational thermal envelopes, and most importantly, constitute "black boxes." When manipulating civic safety equipment, municipal engineers demand interpretability.
- **Decision Tree Supremacy:** The `GradientBoostingRegressor`` (via `scikit-learn``) implements heavily structured ensemble decision-tree mathematics. It handles nonlinear interactions natively (e.g., poor weather

only matters _if_ it is dark) and operates efficiently on tabular data arrays (Time, Visibility, Proxies) without the catastrophic overfitting typical of neural manifolds.

1.4 Deep System Component Architecture Flow

graph TD

```
classDef frontend fill:#3498db,stroke:#2980b9,stroke-width:2px,color:#fff;
classDef backend fill:#9b59b6,stroke:#8e44ad,stroke-width:2px,color:#fff;
classDef database fill:#f1c40f,stroke:#f39c12,stroke-width:2px,color:#333;
classDef hardware fill:#e74c3c,stroke:#c0392b,stroke-width:2px,color:#fff;
classDef machinelearning fill:#2ecc71,stroke:#27ae60,stroke-width:2px,color:#fff;
classDef api fill:#34495e,stroke:#2c3e50,stroke-width:2px,color:#fff;
```

```
subgraph Presentation_Tier [Frontend Layer - Full Stack UI]
```

```
  UI_Browser[User Browser - Dashboard]:::frontend
```

```
  Streamlit[Streamlit Server]:::frontend
```

```
  Folium[Folium GIS Module]:::frontend
```

```
  UI_Browser <-->|WebSockets HTTP| Streamlit
```

```
  Streamlit -->|Generates HTML JS| Folium
```

```
end
```

```
subgraph Edge_Computing_Tier [Edge Hardware Vision Layer]
```

```
  Hardware_Camera[Traffic Camera /dev/video0]:::hardware
```

```
  Thread_Worker[CameraWorker Python Thread]:::hardware
```

```
  YOLO_Engine[YOLOv8 Tensor yolov8n.pt]:::machinelearning
```

```
  RAM_Lock[(Thread-Safe Mutex)]:::database
```

```
  Hardware_Camera -->|RGB Frames 30fps| Thread_Worker
```

```
  Thread_Worker -->|Inference Cycle| YOLO_Engine
```

```
  YOLO_Engine -->|Traffic Risk Score| RAM_Lock
```

```
end
```

```
subgraph Data_Persistence_Tier [Relational DB Layer]
```

```
  SQLite[(kerala_smart_lights.db)]:::database
```

```
  DB_Zones[city_zones Table]:::database
```

```
  DB_Weather[weather_data Table]:::database
```

```
  DB_Metrics[viirs_data Table]:::database
```

```
  DB_Predictions[predictions Table]:::database
```

```
  SQLite --- DB_Zones
```

```
  SQLite --- DB_Weather
```

```
  SQLite --- DB_Metrics
```

```
  SQLite --- DB_Predictions
```

```
end
```

```
subgraph Scientific_Data_Ingestion [External APIs]
```

```
  NASA[NASA LAADS DAAC]:::api
```

```
  OpenMeteo[Open-Meteo REST API]:::api
```

```
end
```

```
subgraph Machine_Learning_Tier [Offline Analytics ML Pipeline]
```

```
  Jupyter[Jupyter Training Pipeline]:::machinelearning
```

```
  SciKit[GradientBoostingRegressor]:::machinelearning
```

```
  PKL_Model[intensity_model.pkl]:::machinelearning
```

```
  NASA -->|Raw Sat Radiance| Jupyter
```

```
  OpenMeteo -->|Historical JSON Weather| Jupyter
```

```
  Jupyter -->|Training Array Synthesis| SciKit
```

```
  SciKit -->|Weights Export| PKL_Model
```

```
end
```

```
Streamlit <-->|Read Write Requests| SQLite
```

```
Thread_Worker -->|Write TRS Events| SQLite
```

```
PKL_Model -->|Loaded at Boot| Streamlit
```

```
RAM_Lock -->|Read Every 1.0s| Streamlit
```

LumiCity is intrinsically divided into three completely separate operational loops operating simultaneously.

Pipeline A: The Analytical & Training Loop

Executed offline natively inside a Jupyter `ipy nb` architecture, this loop handles the raw scientific burden.

1. **Data Ingestion:** Iteratively extracts gigabytes of localized telemetry from NASA EarthData (VIIRS) and Open-Meteo REST parameters.
2. **Safety Target Generation:** Implements human-designed safety heuristics to build the flawless target map (`y`).
3. **Noise Injection:** Artificially corrupts this perfect geometrical mapping with a `numpy` Gaussian blur (= 6%) to prevent pathological tree overfitting.
4. **Binary Serialization:** Dumps the resulting finalized multidimensional mathematical tensor into a raw `intensity\_model.pkl` file structure for future inference consumption.

Pipeline B: The Edge Edge Surveillance Thread (`CameraWorker`)

Running concurrently natively inside the Streamlit instance, this layer performs real-time visual overrides.

1. Extracts individual BGR bitmap frames from the live webcam hardware loop (`/dev/video0`).
2. Passes every N-th frame into the massive `yolov8n.pt` mathematical classification network.
3. Strips non-critical classifications from the output tensor, scoring and multiplying heavy, massive objects (Trucks, Buses) exponentially higher than slow, lightweight frames.
4. Updates a thread-safe dictionary memory space locked behind a `threading.Lock()`, ensuring the UI reading it does not cause catastrophic segmentation fault reading conflicts.

Pipeline C: The Reactive Presentation Pipeline

1. Evaluates the absolute base minute-timer constraints on a perfect physical interpolation curve.
2. Fetches the discrete Gradient Boosting AI prediction vector for the localized node's current temporal alignment.
3. Injects the real-time YOLOv8 `intensity\_boost` factors directly on top of the calculated baseline.
4. Renders the entirety of this mathematical symphony as a visual geographical map mapping to a color-scaled legend (Green = Efficient, Red = Wasteful/Overridden).

Part 2: Relational Database Architecture & Deep Semantic M

2.1 The Philosophy of Localized Persistence

In modern distributed web architecture, persistence layers are heavily abstracted, often spanning multiple physical servers located hundreds of miles away (e.g., AWS RDS, Azure Cosmos).

LumiCity deliberately eschews this pattern. By utilizing SQLite (`kerala\_smart\_lights.db`), the system operates entirely within localized file envelopes. Why? Because a civic street lighting server deployed directly into a municipal telecom closet cannot afford external DNS resolution failures or unhandled REST packet loss when processing 30-frame-per-second traffic camera predictions. The data must reside exactly where the processing matrix is physically located.

This database enforces extremely strict Third Normal Form (3NF) structures, preventing anomalous data duplication and assuring the total structural integrity of the application.

2.2 Deep Schema Structural Analysis

The database consists of precisely highly-specialized interconnected structural tables. Below is a rigorous analysis of their schema definitions, their semantic purpose, and their mathematical significance to the broader AI inference pipeline.

```
erDiagram
    city_zones ||--o{ street_lights : "contains"
    city_zones ||--o{ weather_data : "experiences"
    street_lights ||--o{ viirs_data : "monitored by"
    street_lights ||--o{ predictions : "receives"
    street_lights ||--o{ detection_events : "surveilled by"
    street_lights ||--o{ override_events : "overrides"

    city_zones {
        VARCHAR city_code PK
        FLOAT center_lat
        FLOAT center_lon
    }
    street_lights {
        VARCHAR light_id PK
        VARCHAR city_code FK
        FLOAT latitude
        FLOAT longitude
    }
    weather_data {
        INTEGER id PK
        VARCHAR city_code FK
        FLOAT visibility
        BOOLEAN is_poor_visibility
    }
    predictions {
        INTEGER id PK
        VARCHAR light_id FK
        FLOAT predicted_intensity
    }
```

2.2.1 Geographical Scaffolding: `city\_zones`

This table defines the overarching geographical bounding boxes of municipalities. It is exclusively utilized by the Streamlit frontend to physically construct the spatial rendering matrix for the `folium` maps.

```
CREATE TABLE city_zones (  
  city_code VARCHAR PRIMARY KEY,  
  zone_name VARCHAR,  
  center_lat FLOAT,  
  center_lon FLOAT,  
  south_lat FLOAT,  
  north_lat FLOAT,  
  west_lon FLOAT,  
  east_lon FLOAT  
)
```

- `city\_code`: The absolute relational anchor. A 3-character strict municipal identifier (`TVM`, `KCH`). This acts as the Foreign Key mapping anchor for almost every other operational query.
- The `\_lat` and `\_lon` bounding vectors physically delineate the rectangle overlay structure displayed graphically to the human operator on Tab 1 of the Streamlit dashboard.

2.2.2 The Physical Grid: `street\_lights`

The definitive, immutable registry describing every single structural street light node in the civic network. This table represents the physical grid that the ML model attempts to optimize. If a node is not explicitly defined in this table, it does not mathematically exist to the inference pipeline.

```
CREATE TABLE street_lights (  
  light_id VARCHAR PRIMARY KEY,  
  city_code VARCHAR,  
  latitude FLOAT,  
  longitude FLOAT,  
  location_name VARCHAR,  
  fixture_type VARCHAR  
)
```

- `light\_id`: Complex string identifier linking a zone and node chronologically (e.g., `NODE-TVM-A01`).
- `latitude` and `longitude`: High-precision GPS floats. These are incredibly important. The entire NASA VIIRS skyglow data extraction script utilizes these exact coordinates to bound the 500-meter physical satellite telemetry scan vectors.

2.2.3 The Meteorological Telemetry Envelope: `weather\_data`

Houses the highly granular physical Open-Meteo REST metrics. This represents the immediate atmospheric status above a specific zone. Because weather changes aggressively, this dataset dictates raw safety floor limits before the AI is even consulted.

```
CREATE TABLE weather_data (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  timestamp DATETIME,  
  city_code VARCHAR,  
  cloudcover FLOAT,  
  visibility FLOAT,  
  is_poor_visibility BOOLEAN  
)
```

- `visibility`: Evaluated rigorously in physical meters.
- `is\_poor\_visibility`: A computed boolean truth factor. The fundamental system rule states: If visibility < 3000 meters, `is\_poor\_visibility = TRUE`. Whenever this evaluates to TRUE, the AI's predictive authority is severely handicapped, establishing an absolute unmovable base floor of 60% lighting intensity to guarantee civic traffic safety.

2.2.4 The Scientific NASA Ground-Truth: `viirs\_data`

This table houses the literal orbital space measurements. VIIRS (Visible Infrared Imaging Radiometer Suite) is fundamentally sensitive to the barest trace of terrestrial light emission.

```
CREATE TABLE viirs_data (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  date DATE,
  light_id VARCHAR,
  raw_radiance FLOAT,
  skyglow_index FLOAT,
  quality_flag INTEGER
)
```

- `raw\_radiance`: Ground-truth scalar measurement captured directly from the Suomi NPP orbit, calculated in Watts per square centimeter per steradian (W/cm²/sr). High values directly indicate dangerous, uninhibited localized light pollution spillage.
- `skyglow\_index`: A computational scalar proxy normalized for UI presentation formatting.

2.2.5 Artificial Intelligence Model Telemetry & Audit: `model\_metrics`

An internal auditing matrix heavily updated at the exact moment the Gradient Boosting `scikit-learn` iteration script finalizes in the offline Jupyter architecture. It tracks the chronological precision drift of the AI models.

```
CREATE TABLE model_metrics (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  trained_at DATETIME,
  model_type VARCHAR,
  mae FLOAT,
  rmse FLOAT,
  r2 FLOAT,
  custom_accuracy FLOAT
)
```

- `rmse` (Root Mean Squared Error): By aggressively squaring the residual inference faults, this penalizes models that suffer from wild, unhinged structural hallucinations during training execution.
- `r2` (R-squared): Represents the raw statistical percentage of the variance inside the training dataset that is actively explained by the physical telemetry parameters.

2.2.6 The Core Application State: `predictions`

The primary computational bridge connecting the heavy data engineering inherent in the Jupyter ML pipeline directly to the lightweight Streamlit frontend. The Frontend passively reads this table rather than wastefully attempting to load `.pkl` matrices and recalculating the thousands of decision trees at runtime for every frame render sequence.

```
CREATE TABLE predictions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    light_id VARCHAR,
    target_timestamp DATETIME,
    predicted_intensity FLOAT,
    model_version VARCHAR
)
```

- ``predicted_intensity``: The pure, un-boosted algorithm result (bounded geometrically 0 to 100). This is essentially the Machine Learning model's "vote" on how bright the light `_should_` be at `_precisely_`target_timestamp``.

2.2.7 Real-Time Edge Tensor Detections: ``detection_events``

Stores localized Edge Computing YOLOv8 object detection summaries. Capturing every single frame (30 times a second) natively into sqlite would immediately crash the storage physical disk (IOPS exhaustion). Therefore, the data structure is down-sampled significantly.

- ``trs_max``: The absolute highest peak Traffic Risk Score detected mathematically inside the locked 5-second asynchronous buffer window.
- ``intensity_boost``: The reactive UI augmentation scale mapping (+10, +20, ..., +40).

2.2.8 Civic Command Authority Log: ``override_events``

The absolute safety audit log tracking manual municipal interventions via the Streamlit UI. Every time a city official forces a state, it permanently stamps a record mapping the user, the time, and the exact physical variance forced.

2.3 The Crucial Architecture of Asynchronous Disk Writing

Because SQLite is fundamentally a local file-based lock system relying on the operating system for synchronization, extreme care must be taken during high-frequency multithreading.

Streamlit evaluates at approximately 1 frame every 1.5 seconds to refresh the UI maps.

Simultaneously, the YOLOv8 tensor thread iterates roughly 4-10 times per second in the absolute background.

If both systems trigger ``sqlite3.connect().execute()`` natively at the same microsecond, Python raises the catastrophic ``sqlite3.OperationalError: database is locked`` exception, completely crashing the presentation loop wrapper.

****The Throttling Mechanism:****

To aggressively circumvent this disk-I/O hardware limitation, ``app.py`` enforces temporal bounding logic:

```
`if time.time() - st.session_state["cam_last_db_write"] >= 5.0:`
```

By restricting YOLOv8 database injections to exactly one batch statement every rolling 5000 milliseconds, we preserve the SSD geometry and structurally eliminate read/write locking starvation on the ``kerala_smart_lights.db`` file.

Part 3: Backend External Extractors & Data Engineering

3.1 The Philosophy of Physical Data Extraction

To achieve algorithmic precision vastly superior to synthetic modeling or static mathematical curves, LumiCity abandons internal heuristics and is instead engineered to harvest literal ground-truth measurements from international scientific APIs. This specific architectural concept is termed "Physical Extractor Logging".

We vehemently oppose generating random floating-point values for testing assumptions; the ML models ingest exact historical planetary observations mapped strictly down to the precise decimal longitude and latitude defined in the `street\_lights` relational matrix.

3.2 The NASA LAADS DAAC Telemetry Pager

```
sequenceDiagram
    participant App as LumiCity Backend
    participant Cache as RAM Cache
    participant NASA as LAADS DAAC
    participant Disk as SSD

    App->>Cache: Request VIIRS Data (lat, lon)
    alt Cache Hit
        Cache-->>App: Return cached raw_radiance
    else Cache Miss
        App->>NASA: Authenticate & Request VNP46A1
        NASA-->>Disk: Download 500MB HDF5
        App->>Disk: Parse .h5 (h5py)
        App->>App: Extract Bounding Box & Average
        App->>Disk: os.remove() (Delete Granule)
        App->>Cache: Store result
        App-->>App: Return raw_radiance
    end
```

The Python function `fetch\_nasa\_viirs\_data(lat, lon, date\_str)` represents the most rigorously scientific and computationally heavy component of the telemetry pipeline.

3.2.1 Understanding the VIIRS Instrument

VIIRS (Visible Infrared Imaging Radiometer Suite) is a highly advanced scanning radiometer, carried aboard the Suomi National Polar-orbiting Partnership (Suomi NPP) and NOAA-20 satellite platforms. Unlike standard weather satellites, VIIRS carries a profound engineering achievement: the Day/Night Band (DNB). The DNB is uniquely sensitive to incredibly minimal terrestrial light sources, capable of measuring municipal skyglow, fishing fleet illumination, and localized gas flaring from absolute orbit.

3.2.2 NASA Analytical Extraction Workflow

Extracting scalar telemetry from orbital mechanics is not a simple REST JSON call. It is a multi-step decimation process:

1. **Bounding Box Formulation:** NASA does not afford point-query databases. They store data in massive,

contiguous 500MB+ daily orbital Granules (stored iteratively in the scientific `.h5` HDF5 format). The LumiCity system geometrically generates a bounding box extending exactly 0.05 degrees radially outward from the specific parameter `latitude` and `longitude` of a given street light node.

2. ****API Rest Negotiation:**** An authenticated HTTP GET request is issued directly to the `https://ladsweb.modaps.eosdis.nasa.gov` endpoint. The platform specifically requests `product=VNP46A1` (the Black Marble daily radiance product) from `collection=5000` against the target `date\_str` horizon.
3. ****Streamed Granule Downloading:**** If the remote NASA JSON metadata dictionary confirms a valid HDF5 file intersects our bounding coordinate matrix for that specific temporal day, the system opens a `requests.Session()` with streaming configuration active. It chunk-downloads the entire 500MB payload locally into a temporary file matrix: `viirs\_YYYY-MM-DD.h5`.
4. ****Local HDF5 Matrix Parsing:**** Utilizing the `h5py` Python scientific library, the system breaches the hierarchical data format. It drills down through the nested array dataset: `HDFEOS][GRIDS][VNP\_Grid\_DNB][Data Fields][DNB\_At\_Sensor\_Radiance\_500m`.
5. ****Coordinate Isolation & Radiant Averaging:**** It actively masks and drops erroneous pixel measurements resulting from sensor calibration noise (`rad <= 0` or specifically `rad >= 65535`). It identifies the 4 to 9 localized physical pixels covering our bounding box, geometrically averages their `Valid\_Radiance` (measured strictly in terms of Watts per square centimeter per steradian: W/cm²/sr), and outputs this precise scalar multiplier representing the absolute historical light pollution ceiling directly above the specific street light.
6. ****Cleanup & RAM Caching:**** The 500MB granule is abruptly deleted via `os.remove()` to prevent local storage SSD overwrite exhaustion. The resultant output float is then globally cached in system RAM (`\_viirs\_cache`), guaranteeing that subsequent chronological requests for the same exact night bypass the exhaustive 5-minute payload download phase entirely.

3.2.3 Volatile Network Error Tolerance

Because the NASA endpoint handles petabytes of global scientific requests daily, timeouts are a statistically frequent occurrence-especially given the payload sizes and scientific-tier rate limit architectures.

If the `fetch\_nasa\_viirs\_data` loop exhausts its intrinsic retry matrix, it safely and gracefully falls back to `15.4 W/cm<sup>2</sup>/sr`, a statistically and mathematically sound regional median baseline for urban density in Kerala, India.

3.3 The Open-Meteo Archival Harvester

While NASA's VIIRS provides an absolute measurement of light output potential, the ML algorithm desperately requires exact historical macro-weather parameters to provide context. The `fetch\_real\_weather(lat, lon, start\_date, end\_date)` function fulfills this mandate.

- **_Endpoint Target:** `archive-api.open-meteo.com`
- **_Parameters Requested:** The payload explicitly targets `cloudcover` (as a physical percentage) and `visibility` (as an absolute measurement in meters). Open-Meteo returns serialized 24-hour chronological arrays for the specified date range, isolated perfectly to the provided Node `lat/lon` floating coordinates.
- **_Pandas Re-alignment:** The returned JSON payload is dynamically converted into a `pd.DataFrame`. The Pandas functions `ffill` (Forward Fill) and `bfill` (Backward Fill) imputation are utilized structurally to resolve any transient null measurements or dropped sensor packets in the historical array, guaranteeing the ML

trainer always receives a complete 24-hour tensor slice.

3.4 Geometric Modeling of Human Commuter Traffic

Unlike macro-meteorological data, extracting exact, granular automotive traffic telemetry (e.g., from the Google Maps Distance Matrix API) on a node-by-node, hour-by-hour basis is notoriously, economically prohibitive to harvest. Doing so for a 14-day archival horizon across an entire civic municipality would cost thousands of dollars per month in API charges.

Instead, we algorithmically generate a geometric proxy of the `_surge_` of human commuters using overlapping harmonic sinusoids.

```
`TrafficFactor = MAX(0.1, SIN(pi * (hour - 4) / 12) + 0.5 * COS(pi * hour / 6))`
```

3.4.1 Mathematical Deconstruction of the Proxy

- **The Primary Sinusoidal Construction:** This equation utilizes a primary sine wave phase-shifted geometrically by exactly 4 hours. This mathematically anchors the major morning civic commute as beginning roughly at 4:00 AM, peaking near mid-day solar noon, and gracefully returning toward the baseline post-sunset.
- **The Secondary Evening Harmonic:** The ``0.5 * cos(...)`` appended term injects a smaller, denser evening "second rush hour" bump. This geometrically mirrors the phenomenon of citizens returning from work, increasing street load temporarily between 17:00 and 19:30.
- **The Absolute Zero-Bound Floor:** The entire function is mathematically bounded against ever falling below ``0.1``. A street is never truly empty. There is always the non-zero probability of an ambulance, a night-shift worker, or a police patrol. The ``0.1`` floor represents this structural minimum.

The resultant proxy float value (0.1 to 1.5) is structurally aggregated with the NASA radiation metric to rigidly bound the AI algorithms during standard, non-weather-event nights, providing the algorithm an assumption of "load" without paying for live GPS tracking of citizens.

Part 4: Light Intensity AI Algorithm & Mathematical Formula

4.1 The Fundamental Proposition of Machine Learning

In civic deployment, the core ML prediction loop does `_not_` arbitrarily guess a floating-point integer between 0-100%. The explicit goal of the AI model is to output a rigidly derived ``Predicted Intensity`` score that is precisely reactive to tabular features (Hour, Month, Visibility, Celebration_Multiplier) mathematically mapped to our physical limits defined in Part 3.

4.2 Algorithm Selection: Gradient Boosting Regressor (GBR)

We deliberately selected ``GradientBoostingRegressor`` (GBR) from the Python ``scikit-learn`` framework over more publicized Deep Learning libraries (like TensorFlow or PyTorch).

Sub-symbolic neural networks (like Deep Learning MLPs) obscure their internal feature weights across millions of floating-point parameters, creating highly unpredictable, non-deterministic "black-boxes". This is completely unacceptable for public safety infrastructure like street lighting where legal liability requires mathematical justification for a dimming action.

Decision tree ensembles are vastly more explainable, immune to standard un-scaled distributions, and train significantly quicker on standard CPU hardware.

4.2.1 Ensembling and Gradient Descent

- **Ensemble Construction:** GBR iteratively and chronologically constructs thousands of rudimentary, shallow decision trees. Each new tree mathematically prioritizes minimizing the `_residual errors_` (loss function, specifically Root Mean Squared Error) of the previously appended trees. It learns entirely from its past mistakes.

- **Key Hyperparameters (Implied):** Trees (``n_estimators=100-300``), Tree Depth (``max_depth=3-5``), and Learning Rate (``learning_rate=0.1``). These govern how swiftly the algorithm aggressively "fits" against the weather constraints and traffic proxy equations. Shallow depths intentionally prevent the trees from memorizing the specific noise of the training data, allowing them to generalize beautifully onto completely unseen datasets.

4.2.2 Formulating the Objective "Ideal" Target Function (y)

flowchart TD

```
A[Start Target Engine] --> B{Is it Daytime?}
B -- Yes --> C{Visibility < 3000m?}
C -- Yes --> D["y = min(100, 50 + Proxy)"]
C -- No --> E["y = 0"]
B -- No (Night) --> F["y = Base Night Formula"]
F --> G{Visibility < 3000m?}
G -- Yes --> H["y = max(y, 55.0)"]
G -- No --> I["y = max(y, 40.0)"]
H --> J[Apply Gaussian Noise]
I --> J
```

```
D --> J
E --> J
J --> K[End Vector Loop]
```

In the offline Jupyter Notebook, a highly complex nested `for` loop constructs the physical training matrices (`X`) and the absolute, perfect mathematically ideal targets (`y`). The Gradient Boosting algorithm must learn to reverse-engineer these rigid rules defined intrinsically by our structural laws.

****The Base Training Constraints:****

For the 14-day historical array across all nodes, the loop models the target `y` (the absolute required intensity given 20/20 hindsight of the physical conditions).

1. ****Safety Override Priority (Daytime)****

- If the chronological `Hour` is ≥ 6 AM AND < 6 PM (Standard Daylight):
- Normally, Intensity = 0 (Lights are explicitly OFF).
- However, if the atmospheric parameter Visibility_poor evaluates to TRUE (< 3000 meters or dense monsoon fog):
 $y = \text{MIN}(100, 50.0 + \text{TrafficProxy_nasa})$
- This critical logic ensures the street light matrix ignites dynamically during daytime monsoon storms, completely bypassing the mechanical clock-timer relied on by legacy cities.

2. ****Nighttime Optimization Bounds & Skyglow Reduction****

- If `Hour` corresponds to standard astronomical darkness:
 $y = \text{MIN}(100, (25.0 + \text{TrafficProxy_nasa} * 5.0) * \text{CelebrationMultiplier})$
- Even then, if Visibility_poor is TRUE, we enforce an absolute minimum floor $y = \max(y, 55.0)$ to guarantee road visibility through precipitation.
- ****The Baseline Safety Net:**** Under absolutely perfect, crystal-clear deep-night conditions with zero traffic and zero celebrations (3:00 AM on a Tuesday), the formula bottoms out at $y = \max(y, 40.0)$.
- ****This single boundary is the system's primary weapon against global light pollution.**** A static legacy timer puts out 100% raw wattage across a full 14 hours of night. The AI correctly determines that 40% intensity is optimal, generating massive, compounding kW/h savings while providing structurally sound, legally compliant lumen levels for the empty roadway.

4.2.3 The Gauss-Markov Necessity: Random Noise Injection

During training, you will observe the following line:

```
`intensity += float(np.random.normal(0, 6))`
```

This line inside the Notebook artificially and deliberately injects Gaussian statistical variance (a standard deviation of 6%) into the mathematically flawless target variable y .

Why intentionally corrupt perfect, clean data? Attempting to train a Gradient Boosting model on a deterministically solvable geometric equation results in immediate catastrophic over-fitting ($R^2 = 1.0$). The AI simply learns to be a calculator. By injecting a physical-tier stochastic noise boundary, the model correctly assumes an $R^2 \sim 0.92$. It "learns" to smooth the jagged edges of geometric variables over time, generalizing substantially better against unseen, anomalous data distributions downloaded in the future.

4.3 Production Deployment: Minute-Level Real-Time Interpolation

Once the `.pkl` binary object model generates the daily 24-hour array of predictions into `kerala\_smart\_lights.db`, the Streamlit application performs sub-hourly, minute-by-minute interpolations via the `compute\_base\_intensity` function.

4.3.1 The Algorithmic Time-Curve Fractional Anchor Matrix

The frontend application operates on the strict minute constraint. We cannot theoretically allow a physical LED street light to instantly, jarringly jump from 10% to 50% power exactly at 20:00:00 (this causes severe pupil dilation shock for motorists).

We employ a continuous linear interpolation function bridging the temporal gap smoothly.

```
# Creates a floating-hour proxy (e.g. 19.5 represents exactly 7:30 PM)
t = sim_hour + sim_minute / 60.0

anchors = [
    (0.0, 40), (0.5, 32), (1.0, 20), (3.0, 18), (5.0, 20), (5.5, 10), (6.0, 0),
    (17.5, 0), (18.0, 40), (18.25, 48), (18.5, 55), (19.0, 70), (19.5, 69),
    # ... Continues through 24.0
]
```

For any given fractional time t , we computationally locate the specific anchors bounding it $[t_0, t_1]$ paired with target requirement magnitudes $[v_0, v_1]$. The system applies the core linear interpolation algorithm:

$\text{Base} = v_0 + (v_1 - v_0) * ((t - t_0) / (t_1 - t_0))$

This produces a beautifully smooth, undetectable fading action across the evening twilight sequence.

4.3.2 Deep Blending Factors & Event Multiplication

This base minute-level intensity is then structurally manipulated through a cascade of final checks:

- Visibility Floor Verification:** The code queries the SQLite DB for the recorded weather of the last hour. If the computed parameter `is\_poor\_visibility` is active, $\text{Base} = \max(\text{Base}, 60.0)$, overriding the smooth fade immediately.
- Machine Learning Integration Blending:** For deep-night hours, the discrete array prediction previously derived by the Gradient Boosting model is queried from memory. Rather than instantly snapping to the AI output, we heavily weight the AI but _allow the fractional smooth curve to blend_:
 $\text{FinalBase} = 0.7 * \text{AIPrediction} + 0.3 * \text{AnchoredBase}$
- Civic Date Celebration Validation:** Finally, it queries the dynamic string `sim\_date` against `celebration\_calendar`. If a festival multiplier exists for that geography (e.g., 1.8x), it multiplies the resultant output and caps the float array absolutely against 100%.

This formula mathematically isolates highly random Edge-case civic dynamics, resulting in a smooth, non-disruptive, highly optimized power draw curve across an entire smart grid network in mere fractions of a millisecond.

Part 5: Edge Computing Surveillance (YOLOv8 & OpenCV)

5.1 The Edge Computing Paradigm in Civic Infrastructure

While the heavily structured tabular Gradient Boosting AI determines the optimized baseline lighting intensity, it cannot account for instantaneous, chaotic behavioral shifts (e.g., an isolated late-night traffic accident, a sudden high pedestrian volume outside a movie theater, or an emergency ambulance route).

To react to these chaotic phenomena instantly without overwhelming slow centralized internet databases, LumiCity aggressively utilizes "Edge Computing" principles. We locally load the ``yolov8n.pt`` classification model directly into the application's RAM (Random Access Memory) on the physical device controlling the camera. Edge Computing ensures that if the civic internet connection completely fails, the camera can still physically brighten the street lights using the local network loop.

5.2 Convolutional Neural Networks: The Architecture of YOLOv8

****YOLO**** stands for "You Only Look Once". Before the invention of YOLO architectures, image detection algorithms (like R-CNN) operated via a slow two-step process: first proposing thousands of potential bounding boxes (Region Proposals), and then evaluating each box individually through a convolutional classifier. This was incredibly slow and utterly impossible to run on live video without massive server farms.

YOLO radically changed this by framing object detection as a single regression problem. It divides the camera frame into an $S \times S$ grid. Each grid cell mathematically predicts multiple bounding boxes and confidence probabilities simultaneously using a single pass through a Convolutional Neural Network (CNN).

5.2.1 Real-Time Execution Limitations

Despite YOLOv8 being incredibly optimized (specifically the ``yolov8n`` "nano" weights tier selected here), Computer Vision mathematical evaluation on high-definition video arrays remains notoriously computationally blocking. Running a ``predict()`` tensor inference synchronously `_inside_` a reactive Streamlit application loop will completely, unrecoverably freeze the UI wrapper. The charts, geographical maps, and clocks will simply stop updating while the CPU crunches the Neural Network matrix math.

5.3 Asynchronous Threading Execution Models

To counter this catastrophic architectural flaw, LumiCity strictly segregates the Computer Vision capability from the User Interface using Python's native ``threading`` libraries enclosed within a custom wrapper: the ``CameraWorker`` class.

5.3.1 The ``CameraWorker`` State Machine

- ****The ``__init__`` constructor:**** Establishes the ``_stop_event`` flag, the absolutely critical ``_lock`` thread-safety boolean, and strictly normalizes the incoming OpenCV matrix stream to a ``640x480`` pixel constraint.

Bounding the geometry is imperative to prevent arbitrary memory spikes if an engineer plugs in a 4K resolution camera.

- **The Throttling Integer (`infer\_every`):** To drastically improve CPU cycle times and drop the required thermal envelope of the processor, bounding boxes are `_not_` mapped 30 times a second. By setting ``infer_every` = 4`, the tensor is mathematically evaluated approximately ~7.5 times a second. Because cars do not teleport, 7 frames a second is more than adequate to perfectly track vehicular bounding boxes, while instantly returning 75% of the CPU's baseline power back to the Streamlit UI.

- **The ``_run()`` loop:** Invokes the asynchronous video stream loop directly inside a protected isolated pointer space.

5.4 Traffic Risk Score (TRS) Calibration Weighting

flowchart LR

```
A(cv2.VideoCapture) -->|Frame Extract| B(YOLOv8n Tensor)
B -->|Bounding Intersects| C(Filter Objects)
C -- Person --> D[+1.0 TRS]
C -- Car --> E[+2.0 TRS]
C -- Truck/Bus --> F[+3.5 TRS]
D --> G(Sum Total TRS)
E --> G
F --> G
G --> H{trs >= Threshold?}
H -- Yes --> I[Assign Boost % to State]
H -- No --> J[Boost = 0]
```

The core functional innovation in the Computer Vision module is translating basic structural bounding boxes into literal, localized dynamic danger probability matrices.

The YOLO network identifies floating-point integers representing classical COCO (Common Objects in Context) IDs. We explicitly strip irrelevant objects from the matrix (kites, dogs, coffee cups) and heavily isolate civic, mass-based vehicles.

```
COCO_WEIGHTS = {0: 1.0, 1: 1.2, 2: 2.0, 3: 1.5, 5: 3.5, 7: 3.5}
COCO_LABELS = {0: "Person", 1: "Bicycle", 2: "Car", 3: "Motorcycle", 5: "Bus", 7: "Truck"}
```

5.4.1 Kinetic Energy Logic Translation

Every inference cycle loops endlessly across the ``results.bboxes`` tensor output, mapping the raw integer ID to the weighted severity table. This weighting is strictly derived from Classical Mechanics (Kinetic Energy $K = 0.5 * mv^2$ and stopping distance correlations).

- Standard Vehicles:** A standard sedan ``Car`` directly translates to a baseline +2.0 TRS factor.
- High-Mass Vectors:** High-mass kinetic objects (A ``Truck`` or ``Bus``) translate immediately to +3.5. Due to their vastly reduced halting velocity curves and immense momentum, they theoretically require vastly more illumination to brake safely in an emergency.
- Vulnerable Subjects:** Direct human subjects (``Person``) trigger the smallest baseline factor +1.0.

Every single valid bounding box drawn on the frame has its structural weight summed into a total moving scalar float tracker: ``trs += COCO_WEIGHTS[cls]``.

Once frame processing is rigorously completed, an immediate algorithmic staircase calculates the instantaneous required baseline boost percentage for the street light UI array:

```
if trs >= 15.0: boost = 40.0    # Extremely heavy traffic / Traffic Jam
elif trs >= 10.0: boost = 30.0 # Dense Flow
elif trs >= 5.0: boost = 20.0  # Moderate flow
elif trs >= 2.0: boost = 10.0  # Sporadic interaction
else: boost = 0.0              # Empty street; revert immediately to AI baseline
```

5.4.2 Resolving Python Race Conditions (`threading.Lock`)

Because the Streamlit UI and the `CameraWorker` are completely decoupled, they are effectively running blindly alongside one another. If the YOLO loop attempts to overwrite the `frame\_rgb` numpy array in RAM at the exact millisecond Streamlit attempts to read it to draw the screen, Python generates a Memory Error and crashes sequentially.

```
with self._lock:
    self._state["frame_rgb"] = frame_rgb
    self._state["trs"] = trs
```

The `with self.\_lock` structural wrapper ensures absolute Mutex (Mutual Exclusion). It tells the Python Kernel: "While YOLO is updating these variables, entirely block any other part of the application from attempting to read them until I am finished." This takes microseconds but guarantees absolute mathematical stability.

Part 6: Presentation Layer & Civic Interface (Streamlit)

6.1 The Engineering Choice of Streamlit over SPA Frameworks

For civic, municipal-tier internal administration panels, engineers traditionally debate adopting React, Vue, or Angular Single Page Applications (SPAs). However, these massive frameworks possess a rigid, unyielding requirement for complex REST or GraphQL API middleware scaffolding (such as Django REST Framework or FastAPI) to simply view database records. This creates enormous architectural code-bloat for internal tools.

Streamlit was actively selected for LumiCity due to its `_native, seamless Python integration_`. It grants direct, untethered access to Pythonic ML components (like `.pkl`` scalar regression weights), concurrent hardware threading models (OpenCV camera connections), and immediate SQLite rendering queries `_without_` suffering the latency overhead of JSON serialization and HTTP routing delays.

6.2 Structural Authentication and Hashing Security

Because Streamlit natively lacks localized stateless JWT (JSON Web Token) middleware out-of-the-box, LumiCity bypasses this limitation by utilizing Python's deep cryptography libraries injected directly into ``st.session_state``.

```
creds = load_credentials()
hashed = hashlib.sha256(password.encode()).hexdigest()
if username in creds and creds[username] == hashed:
    st.session_state["authenticated"] = True
```

This SHA-256 cryptographic hashing ensures that even if a malicious actor successfully downloads ``credentials.json`` directly from the server structure, they cannot immediately reverse-engineer the municipal passwords.

Upon a successful hash matching block, the application permanently stores the active user's ``city_code`` geographical designation in RAM. A registered superuser (designated purely as ``"admin"``) receives structural dropdown UI privileges allowing them to organically swap ``city_code`` flags at runtime without restarting the application, dynamically reloading the SQL dataset queries.

6.3 Global Metric State & Dynamic CSS Styling DOM Overrides

Streamlit's default rendering style is somewhat rigidly designed toward academic papers. To invoke a futuristic, high-tech command center feel appropriate for a Smart City node, the ``app.py`` script heavily utilizes CSS DOM injection hacking via the ``st.markdown(..., unsafe_allow_html=True)`` parameter.

Classes such as ``intensity-card`` and ``override-active`` construct incredibly smooth gradient background cards (``#1a1a2e`` to ``#00e5ff``) housing massive bold typography headers.

6.3.1 The Real-Time "Skyglow" Ecological Metric

A primary metric displayed atop the massive dashboard is intuitively labeled "Avg Skyglow". This isn't merely an arbitrary static graphic or placeholder-it is dynamically computed structurally on `_every` single render cycle_:

1. The layout logic dynamically gathers the absolute real-time `_Required Intensity_` values of all currently active node fixtures natively from the state dictionary ``light_intensities``.
2. It actively queries the specific mathematical average VIIRS physical radiance baseline for the explicitly selected city: ``SELECT AVG(raw_radiance)...``.
3. It mathematically calculates the derived ecological impact vector: ``sky_val = avg_radiance * (avg_intensity / 100.0)``.
4. If the value physically breaches standard acceptable localized urban limits (e.g. ``> 1.5``), the HTML span structurally injects a critical red hex font (``#f44336``), alerting operators immediately to severe, ongoing ecological damage vectors over that ward.

6.4 Tab 1: Geographical GIS Map Network (Folium integration)

The structural, interactive rendering of nodes inherently utilizes the CartoDB Dark Matter CSS tile set via the ``folium`` module.

- **Zone Bounding Renderings:** Database structural bounds generate massive, translucent (``#1e88e5``) blue rectangular bounding structures geographically overlaying the Folium element to clearly delineate civic jurisdiction lines.
- **Coordinate Iterative Plotting:** Every row inside ``street_lights`` seamlessly translates to an interactive ``CircleMarker``.
- **Stateful Topology Coloring:** Instead of relying on static icon colors, the nodes literally glow on the map strictly based on their active ML state:
 - **Green:** Highly optimized, low power draw (``<40%``).
 - **Orange:** Mid-tier operational state (``<70%``).
 - **Red:** Extreme physical draw (approaching 100%).
 - **Purple (``#9c27b0``):** Indicates a "Structural Hijack" - a civic engineer's manual override is actively disabling the ML prediction array for that localized point.

6.5 Tab 3: Analytical Image Structural Inspection (Light Pollution Profiler)

A highly specialized, completely decoupled diagnostic tab allows civic engineers to upload direct field photography (PNG/JPG payloads) taken by citizens or city workers.

1. The arbitrary image byte stream buffer is directly converted via OpenCV bindings (``cv2.imdecode``) and violently stripped of color to form a localized Grayscale mathematical geometric matrix.
2. Deep numerical numpy computations determine structural density parameters across millions of pixels instantly: Average Total Brightness, Structural Variance (``np.std``), and harsh glare-pixel thresholding (``np.sum(gray > 200)``).
3. These values are rigorously and legally run against international scientific danger guidelines. If the structural over-exposure percentages physically breach a 75% bounds, a severe ecological warning box is generated, alerting municipal operators to dispatch field workers and physically adjust hardware baffles or

disrupt the local firmware logic.

6.6 Tab 4: Environment & Macro-Electrical Analytics

A rigidly strict power calculation proxy gives municipal auditors completely direct, real-time feedback on their exact electrical budgets.

Assuming a baseline benchmark operational draw of `0.12 kW` per physical LED-matrix fixture:

``power_kw = total_lights * (avg_intensity / 100.0) * 0.12``

Corresponding environmental metrics calculate strict carbon offset reductions. By mapping kilowatts to kilograms of CO2 algorithmically, it dynamically tracks the reduction footprint across a standard 12-hour chronological twilight envelope.

This offers immediate, heavily quantifiable Return-On-Investment (ROI) visual tracking, which is absolutely mandatory when justifying civic infrastructure tax allocations and securing state-level climate-action funding grants.

Part 7: Structural Code Deep Dive (app.py)

The `app.py` script serves as the monolithic operational execution engine driving the presentation, ML inference, and heavy Edge-execution systems all simultaneously within a single running instance. To properly understand the architecture, software engineers must execute a structural deep-dive into its core classes, threading mechanisms, and function decorators.

7.1 Import Topography and Bootstrap Matrix

The script immediately imports standard visualization arrays (`streamlit`, `pandas`, `plotly.graph_objects`, `folium`, `streamlit.components.v1`, `streamlit_folium`) alongside deep-computation parameters (`threading`, `cv2`, `numpy`, `json`, `hashlib`, `ultralytics`).

```
import streamlit as st
import sqlite3
import pandas as pd
import datetime
# ...
import numpy as np
from ultralytics import YOLO

st.set_page_config(
    page_title="LumiCity Kerala",
    layout="wide",
    page_icon="",
    initial_sidebar_state="collapsed"
)
```

Upon execution, it instantly defines global application constants mapping 3-character database primary-key tokens (`TVM`, `KCH`) into structural civic strings (`Thiruvananthapuram`, `Kochi`).

Crucially, it utilizes the Streamlit decorator `@st.cache_resource` to violently lock the heavy `yolov8n.pt` Object Detection weights into a persisting RAM cache. If this decorator was missing, the application would structurally attempt to load the multi-megabyte tensor back into memory every single time a user clicked a button in the UI, resulting in horrific 10-second latency spikes.

7.2 The Asynchronous Edge Engine (`CameraWorker`)

This class structurally solves one of the oldest problems in graphical UI development: thread bottlenecking during highly complex 3D array matrix mathematics (tensor operations).

```
class CameraWorker:
    def __init__(self, camera_index=0, width=640, height=480, infer_every=4):
        # Initializes local state, locks, and thread pointers
        self._state = { "frame_rgb": None, "trs": 0.0, "boost": 0.0, "counts": {}, "error": None }
```

When the operator presses "Start Live Feed", `.start()` is invoked. The private method `__run()` completely bypasses Streamlit's structural top-down execution sequence. It spins up a true, infinite `cv2.VideoCapture` `while` loop deep inside a decoupled OS thread.

To computationally prevent the server from overheating and throttling the CPU cores, the actual heavy tensor

matrix block ``model(frame, conf=0.40, iou=0.45)`` explicitly only executes its math if ``frame_idx % self.infer_every == 0``. It structurally skips frames.

It geometrically isolates critical boxes (``if cls in COCO_WEIGHTS``), draws strict bright-green bounding frames bounding the object (``cv2.rectangle``), and algebraically computes the aggregated Continuous Traffic Risk index (``trs``).

Most critically, as discussed in prior chapters, the resulting state map is acquired entirely using a ``with self._lock:`` context manager statement to absolutely prevent race conditions.

7.3 Decoupled Data Access SQL Bridges

sequenceDiagram

```
participant UI as Streamlit Layout Loop
participant Thread as CameraWorker Thread
participant DB as SQLite3 Database
```

```
UI->>DB: query_db(SELECT weather)
DB-->>UI: pandas.DataFrame
UI->>DB: query_db(SELECT prediction)
DB-->>UI: pandas.DataFrame
```

```
Thread->>DB: INSERT INTO detection_events
note over Thread,DB: Hardware Throttled Sequence:<br>1 Write Transaction / 5.0 Seconds
```

Instead of deploying heavy, schema-locked Object-Relational Mappers (ORM) like SQLAlchemy or the Django ORM syntax, ``app.py`` operates significantly "closer to the metal." It utilizes direct structural SQL injection execution wrappers.

```
def query_db(q, params=()):
    conn = sqlite3.connect(DB_NAME)
    df = pd.read_sql_query(q, conn, params=params)
    conn.close()
    return df
```

This forces the SQLite native tuple return vectors strictly and immediately into complex ``pandas.DataFrame`` structures. This natively eliminates iterating over vanilla Python arrays and directly enables rapid analytical graphing algorithms (``plotly``) downstream, saving highly valuable CPU cycles and dramatically reducing memory bloat.

7.4 The AI Interrogator Context (``compute_base_intensity``)

This complex function acts as the final arbiter. It forcefully attempts to resolve immensely conflicting algorithmic constraints arriving from simple minute-timers, hardcoded structural overrides, external weather models, and predictive ML Dataframe metrics.

```
# 1. Fractional Spline Anchor Interpolation
t = sim_hour + sim_minute / 60.0
# ...
```

1. **It first interpolates fractional bounding anchor arrays** to resolve smooth LED dimming constraints over time.
2. **It then executes a deep meteorological override SQL validation:** ``SELECT is_poor_visibility FROM weather_data``.

3. **Following this, it queries the pure Artificial Intelligence array** (`predicted_intensity`) for the precisely matching chronological integer ID linking strictly to `sim_hour`.
4. **Mathematical Blending Matrix:** If the UI state reflects astronomical darkness, it weights the Neural tree output tightly against the simpler structural clock timer via coefficients (`0.7 * ai_val + 0.3 * base`).
5. **Multiplication Check:** Finally, it runs a hard string comparison check against `celebration_calendar` to scale the scalar value abruptly upward if unique, transient local festival logic dictates.

7.5 Stateless Dictionary Navigation & Override Event Handling

Due to its architecture, Streamlit is fundamentally stateless; it re-executes the global Python script from the top-down upon every slider or button interaction. The application must anchor data firmly in `st.session_state` to establish the application's persisted dynamic RAM envelope.

```
if "overrides" not in st.session_state:
    st.session_state["overrides"] = {} # Defines {"mode": "MANUAL", "value": int}
```

When a local civic engineer selects "MANUAL Override - 80%", the dictionary catches the Node ID string key representing that specific street light and forcefully commands the primary render loop to wholly bypass `compute_base_intensity()`. Concurrently, it writes the exact chronological interaction matrix directly to `INSERT INTO override_events` to enforce strict civic auditing protocol.

7.6 The Real-Time Polling Strategy (`@st.fragment`)

To update the video element without tearing down and redrawing the huge Folium maps every single second:

```
@st.fragment(run_every=1.0)
def camera_feed():
    snap = worker.snapshot()
    trs = float(snap["trs"])
    boost = float(snap["boost"])
    final_i = min(100.0, base_i + boost)
    st.image(snap["frame_rgb"], caption=" LIVE stream", use_container_width=True)
```

The recently released `@st.fragment` decorator mathematically isolates exactly this segmented block of Python logic from the primary HTTP thread pipeline. The code evaluates the boolean payload state of the camera lock buffer strictly every `1.0` seconds on an infinite mechanical loop.

It instantly calculates the absolute `final_i` intensity voltage requirement (`AI_Baseline + Live_CV_Boost`), and structurally updates the localized CSS DOM rendering tag on the client-side browser-completely without triggering a structural total page refresh and wiping user parameters.

Part 8: Model Training Logic & Pipeline Diagnostics (Jupyter)

8.1 Data Engineering Execution Summary

The ``LumiCity_Step_by_Step_Analysis.ipynb`` module serves not as active presentation UI, but purely as the offline, computationally heavy data engineering, algorithm training, and statistical matrix normalization environment. This must be successfully executed `_before_` any ML weights are permitted to be injected into the production architecture.

It elegantly leverages the vast mathematical ``scikit-learn`` framework to permanently formalize the highly non-linear, multi-dimensional relationships connecting NASA VIIRS space telemetry datasets, localized Open-Meteo atmospheric restrictions, and distinct chronological commuter patterns.

8.2 Matrix Array Feature Engineering & Synthesis

The execution framework loops forcefully over a vast, highly exhaustive ``14-day`` retroactive historical data window. It invokes strict remote HTTP Socket structures to continuously pull JSON parameter telemetry blocks spanning thousands of individual street light locations.

The resultant Pandas output DataFrame (``Data Matrix assembled: roughly 23,520 physical integer data points``) directly constructs the primary learning tensor axis ``X``. The features are:

- `**`hour` Parameter:**` Geometric floating interval 0 to 23.
- `**`day_of_week` Offset:**` Represents standard tracking 0 to 6 (weekend designations actively push massive peak traffic loads chronologically outward into the night, delaying baseline intensity decay curves).
- `**`month` Scalar:**` Captures the deep transient temporal season shift (vital in India due to distinct Monsoon behaviors significantly altering daily cloud refraction indices).
- `**`is_poor_visibility` Boolean:**` Maps strongly to 0 or 1. This acts as the absolutely most critical feature weight algorithmically. If visibility drops below a hard 3-kilometer index, the AI is structurally taught to force lighting upwards regardless of empty roadways.
- `**`viirs_radiance` Space Float:**` The absolute physical background light constraint extracted locally from HDF5 NASA mapping arrays via our caching protocols.
- `**`traffic_proxy` Calculation:**` The beautifully derived structural human-commuter load parameter mapped geometrically via dual harmonic sine functions, preventing zero-bound AI drops.
- `**`celebration_mult`:**` 1.0 (baseline) or heavily > 1.0 , corresponding iteratively to massive civic crowd festival dates plotted over the geometry.

8.3 Algorithmic Target Array Generation & Statistical Preprocessing

The model training loops explicitly map out the theoretical ``required_intensity`` baseline strictly based on the structural safety frameworks documented in previous chapters.

Gaussian randomization injection (``numpy.random.normal(0, 6)``) deliberately completely blurs and scrambles the mechanically perfect vector mapping. This prevents standard, devastating $R^2 = 1.0$ data-leakage

overfitting faults. It violently forces the Gradient Boosting tree ensemble to synthesize highly robust, profoundly generalized mathematical multidimensional decision boundaries, rather than ignorantly memorizing a pure static algebraic syntax script payload.

The mathematical arrays are essentially subsequently split strictly using a 80/20 train/test verification threshold:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Standardization structural execution (`StandardScaler`) forcibly centers the overall dataset variance distributions heavily around a `0` mean with a standard deviation curve of `1`. This incredibly significantly accelerates the deep mathematical gradient descent vectoring phase execution sequence, ensuring the model reaches convergence minima exponentially faster.

8.4 Mathematical Model Compilation & Robust Metric Output Iteration

```
`model      =      GradientBoostingRegressor(n_estimators=200,      learning_rate=0.1,      max_depth=4,
random_state=42)`
```

The Jupyter notebook kernel calls the massive computation block `model.fit(X\_train\_scaled, y\_train)` mapping roughly 18,000+ points accurately across millions of individual fractional decision splines simultaneously.

Upon computational completion, the AI system rigorously constructs diagnostic regression metrics allowing the civic engineer to verify the structural integrity of the learning attempt:

- **Mean Absolute Error (MAE):** Measures the literal absolute structural percentage deviation of the Artificial Intelligence's selection output versus the perfect physical ground-truth ideal setting parameters calculated earlier safely.
- **Root Mean Squared Error (RMSE):** Imposes a much heavier geometric logarithmic penalization curve expressly against drastic, catastrophic algorithmic hallucinations that could severely endanger daytime drivers.
- **R-squared Index (R^2):** Analytically explains the deep internal variance proportion mapping tracking across matrices. Given our deliberately implemented Gaussian blur noise models on the target arrays, this successfully and correctly floats beautifully near the 0.90 to 0.94 bracket bounds.
- **Custom Safety Accuracy Threshold (CSAT):** Uniquely measures the absolutely precise calculated percentage of sequential AI output derivations that permanently manage to stay locked within a stringent physical boundary error envelope of merely +/-5% of the physical required safety ceilings.

8.5 Production Object Serialization & Automated Deployment Strategy

```
with open('intensity_model.pkl', 'wb') as f: pickle.dump(model, f)
with open('intensity_scaler.pkl', 'wb') as f: pickle.dump(scaler, f)
```

The deeply complex computational Gradient Boosting tree weights matrix structure and the critical multi-dimensional scaling logic objects are physically and permanently binary-serialized strictly using the

Python `pickle` package structure.

The background architecture subsequently operates automatically via a continuous looping SQL data-injection statement command structure (`INSERT INTO predictions...`). This specifically guarantees that the separate production Streamlit civic presentation architecture immediately benefits from extracting the freshly derived regression structural parameters directly out of the SQLite matrix without any manual software shutdown updates, application server rebuilding cycles, or complex CI/CD pipeline script compiling structures.

Part 9: Core Codebase Reference Appendix

This appendix documents the current production codebase of the LumiCity dashboard application (`app.py`), totaling 1,520 lines of Python code.

Appendix A.1: Module Imports and Configuration

```
import streamlit as st
import sqlite3
import pandas as pd
import datetime
import os, time, json, hashlib, threading
import folium
from streamlit_folium import st_folium
import plotly.graph_objects as go
import streamlit.components.v1 as components
import cv2
import numpy as np
from ultralytics import YOLO
import requests
import pickle
from math import pi

_DIR = os.path.dirname(os.path.abspath(__file__))
DB_NAME = os.path.join(_DIR, "kerala_smart_lights.db")
CREDENTIALS_FILE = os.path.join(_DIR, "credentials.json")
MODEL_PKL = os.path.join(_DIR, "intensity_model.pkl")
SCALER_PKL = os.path.join(_DIR, "intensity_scaler.pkl")
REFRESH_INTERVAL_SECONDS = 24 * 60 * 60
WEATHER_REFRESH_SECONDS = 5 * 60 * 60
```

Appendix A.2: API Extractor Functions

fetch_real_weather(lat, lon, start_date, end_date)

Calls Open-Meteo Archive API for hourly cloudcover and visibility data. Returns a pandas DataFrame with columns: time, cloudcover, visibility. Falls back to empty DataFrame on error.

fetch_nasa_viirs_data(lat, lon, date_str)

Fetches VIIRS VNP46A1 night-time radiance data from NASA LAADS DAAC. Downloads HDF5 granules, extracts DNB_At_Sensor_Radiance_500m, and returns the regional median radiance value. Results are cached in `\_viirs\_cache` dictionary. Falls back to 15.4 W/cm2/sr on error.

get_traffic_factor(hour)

Computes a sinusoidal traffic proxy combining a primary sine wave (morning peak) and secondary cosine wave (evening rush). Returns MAX(0.1, factor). Used to convert raw radiance into traffic-weighted features.

Appendix A.3: CSS Theme (Glassmorphism)

The dashboard uses extensive custom CSS injected via `st.markdown(unsafe\_allow\_html=True)`:

- ``.glass-card``: Main panel style with ``.rgba(17,25,40,0.75)`` background, backdrop blur, rounded corners
- ``.intensity-card``: Street light intensity display cards with dark gradient backgrounds
- ``.cam-result-card``: Camera surveillance result panels
- ``.metric-card``: Analytics metric display with glass effect
- ``.login-container``: Centered authentication form with gradient accents
- ``.header-bar``: Transparent header to prevent UI overlap
- Animated status dots: ``.@keyframes pulse`` for breathing effect on status indicators
- Tab styling: Custom CSS for Streamlit tab buttons with cyan active state

Appendix A.4: CameraWorker Class

Background daemon thread for real-time YOLOv8 traffic surveillance:

```
class CameraWorker:
    def __init__(self, camera_index=0, width=640, height=480, infer_every=4):
        # Thread-safe state via threading.Lock
        # Stores: frame_rgb, trs, boost, counts, error, last_update

    def start(self):      # Launch daemon thread
    def stop(self):       # Signal thread to exit, join with timeout
    def is_running(self): # Check thread health
    def snapshot(self):   # Thread-safe state read for UI

    def _run(self):
        # Opens cv2.VideoCapture with DirectShow backend
        # Every 4th frame: runs YOLOv8n inference
        # Computes TRS from COCO class weights
        # Draws bounding boxes on frame
        # Updates thread-safe state
```

Appendix A.5: DataRefreshWorker Class

Background daemon thread for automatic data refresh with dual cycles:

```
class DataRefreshWorker:
    def __init__(self, interval=86400, weather_interval=18000):
        # State: last_refresh, last_weather, status, error, predictions_count
        # Reads initial timestamps from database in constructor

    def _run(self):
        # On startup: check staleness, refresh immediately if needed
        # Main loop every 60 seconds:
        #   - Check predictions staleness (24h) --> _do_full_refresh()
        #   - Check weather staleness (5h) --> _do_weather_refresh()

    def _do_weather_refresh(self):
        # Fetch Open-Meteo weather for today
        # Update weather_data table for all city codes
        # Log to system_log

    def _do_full_refresh(self):
        # Load model.pkl + scaler.pkl
        # Fetch weather + VIIRS data
        # Generate 1,680 predictions (70 nodes x 24 hours)
        # Replace predictions table
        # Log to system_log
```

Appendix A.6: Database Functions

```
def query_db(q, params={}):
    conn = sqlite3.connect(DB_NAME, timeout=10)
    conn.execute("PRAGMA journal_mode=WAL")
    df = pd.read_sql_query(q, conn, params=params)
    conn.close()
    return df

def insert_db(q, params={}):
    conn = sqlite3.connect(DB_NAME, timeout=10)
    conn.execute("PRAGMA journal_mode=WAL")
    conn.cursor().execute(q, params)
    conn.commit()
    conn.close()
```

Appendix A.7: Intensity Computation Function

```
def compute_base_intensity(light_id, sim_hour, sim_minute=0,
                           sim_date=None, city_code=None):
    # 1. Minute-level interpolation across 21 anchor points
    # 2. Weather safety override (poor visibility >= 60%)
    # 3. AI prediction blend (70% AI + 30% time-curve)
    # 4. Celebration multiplier (capped at 100%)
    return round(base, 1)
```

Appendix A.8: Authentication System

```
def check_password():
    # Glassmorphism login form
    # Municipality selection (8 Kerala cities)
    # SHA-256 password hashing against credentials.json
    # Admin users get city-switching privileges
```

Appendix A.9: Dashboard Layout Structure

```
app.py Layout (top to bottom):
-----
1. Login Screen (if not authenticated)
2. Admin City Switcher (if admin user)
3. City Header + Logout Button
4. Status Bar (components.html iframe):
   - Live Clock | VIIRS Sync | Skyglow | Nodes | Weather | WX Sync | Predict
5. Time Simulation Expander
6. Five Tabs:
   Tab 1: City Map (Folium) + Intensity Cards Grid
   Tab 2: Camera Surveillance + Override + 24h Forecast
   Tab 3: Image Analysis (Upload + Heatmap + Danger Level)
   Tab 4: Energy Analytics + Model Performance Table
   Tab 5: Civic Calendar + VIIRS Data + Add Event Form + Logs
```

Appendix A.10: Database Schema

```
-- 10 tables in kerala_smart_lights.db

CREATE TABLE street_lights (
    light_id TEXT PRIMARY KEY, city_code TEXT, zone_id TEXT,
    latitude REAL, longitude REAL, location_name TEXT,
```

```

    wattage INTEGER, pole_height REAL, installation_date TEXT
);

CREATE TABLE predictions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    prediction_made_at TEXT, target_timestamp TEXT,
    light_id TEXT, predicted_intensity REAL,
    confidence_low REAL, confidence_high REAL,
    day_type TEXT, celebration_multiplier REAL,
    weather_override INTEGER, model_version TEXT
);

CREATE TABLE weather_data (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    city_code TEXT, timestamp TEXT,
    cloudcover REAL, visibility REAL,
    is_poor_visibility INTEGER, fetched_at TEXT
);

CREATE TABLE viirs_data (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    date TEXT, light_id TEXT,
    raw_radiance REAL, skyglow_index REAL, quality_flag TEXT
);

CREATE TABLE model_metrics (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    trained_at TEXT, model_type TEXT,
    mae REAL, rmse REAL, r2 REAL, custom_accuracy REAL
);

CREATE TABLE celebration_calendar (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    event_date TEXT, event_name TEXT,
    multiplier REAL, city_code TEXT
);

CREATE TABLE city_zones (
    zone_id TEXT PRIMARY KEY, city_code TEXT,
    center_lat REAL, center_lon REAL,
    north_lat REAL, south_lat REAL,
    east_lon REAL, west_lon REAL
);

CREATE TABLE detection_events (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    minute_ts TEXT, light_id TEXT,
    trs_avg REAL, trs_max REAL,
    intensity_boost REAL, final_intensity REAL
);

CREATE TABLE override_events (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    ts TEXT, light_id TEXT, user TEXT,
    previous_value REAL, new_value REAL, override_type TEXT
);

CREATE TABLE system_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    ts TEXT, level TEXT, component TEXT, message TEXT
);

```

Appendix A.11: Key Constants

VALID_CITIES:

TVM = Thiruvananthapuram KCH = Kochi

KZD = Kozhikode	TCR = Thrissur
KLM = Kollam	KNR = Kannur
ALP = Alappuzha	PKD = Palakkad

COCO_WEIGHTS (YOLOv8 object class weights):

Person = 1.0	Bicycle = 1.2	Car = 2.0
Motorcycle = 1.5	Bus = 3.5	Truck = 3.5

REFRESH_INTERVAL_SECONDS = 86400 (24 hours)

WEATHER_REFRESH_SECONDS = 18000 (5 hours)

TRS to Intensity Boost mapping:

TRS >= 15.0	-->	+40%
TRS >= 10.0	-->	+30%
TRS >= 5.0	-->	+20%
TRS >= 2.0	-->	+10%
TRS < 2.0	-->	+0%

Part 10: Dashboard Interface - Detailed Screen-by-Screen Walkthrough

10.1 Overview of the Streamlit Dashboard

The LumiCity Dashboard is a single-page Streamlit application (`app.py`) with a modern glassmorphism dark theme. It uses a custom CSS framework with translucent card elements, animated status indicators, and the Inter font family. The application is structured into five primary tabs after an authentication screen.

10.1.1 Visual Design Language

The dashboard employs a carefully crafted visual language:

- **Background:** A multi-stop dark gradient (`#0a0e17` to `#0d1321` to `#111827``) providing depth
- **Glass Cards:** Semi-transparent panels (`rgba(17, 25, 40, 0.75)`) with backdrop blur effects and subtle white borders
- **Accent Color:** Cyan (`#00e5ff`) used consistently for highlights, badges, and gradient text
- **Status Indicators:** Animated pulsing dots (green, cyan, orange, red) conveying system health at a glance
- **Typography:** Inter font (300-900 weights) for clean, modern readability

10.2 Login Screen

The entry point to the application is a centered authentication form with glassmorphism styling.

Layout:

- A centered container (`max-width: 420px`) with glass card styling
- Title: "LumiCity" in large gradient text (cyan to blue)
- Subtitle: "Kerala Smart City Portal" in muted gray
- Municipality dropdown: Lists all 8 Kerala cities (Thiruvananthapuram, Kochi, Kozhikode, Thrissur, Kollam, Kannur, Alappuzha, Palakkad)
- Username and Password fields
- Login button with cyan accent color

Authentication: Credentials are stored in `credentials.json` with SHA-256 hashed passwords. Admin users get additional city-switching privileges after login.

10.3 Header Bar and Status Strip

After login, the top of the dashboard displays:

10.3.1 City Name Header

- City name in bold white (e.g., "Thiruvananthapuram")
- "SMART CITY" badge in a cyan pill-shaped container
- Logout button on the far right

10.3.2 Status Bar (Single `components.html` iframe)

A horizontal bar with real-time system indicators:

- **Live Clock:** JavaScript-driven HH:MM:SS IST display, updating every second
- **VIIRS Sync:** Green pulsing dot + "OK" when satellite data is synchronized
- **Skyglow Index:** Dynamically computed value with color coding:
 - Green (<0.8): Environmentally safe
 - Orange (0.8-1.5): Moderate concern
 - Red (>1.5): Severe light pollution
- **Nodes Count:** Number of active street light fixtures in the selected city (e.g., "70")
- **Weather Status:** Current atmospheric condition:
 - Green dot + "CLEAR": Normal visibility (>3000m)
 - Red dot + "OVERRIDE": Poor visibility triggering safety override
- **WX Sync:** Weather data freshness indicator (5-hour refresh cycle):
 - Green dot + time: Recently fetched
 - Cyan dot + "Updating...": Fetch in progress
 - Orange dot + "Never": Not yet fetched
- **Predict:** Prediction freshness indicator (24-hour refresh cycle):
 - Green dot + time: Recently generated
 - Cyan dot + "Syncing...": Generation in progress
 - Orange dot + "Never": Not yet generated
 - Red dot + "Error": Last refresh failed

10.4 Time Simulation Panel

A collapsible expander allowing operators to simulate future or past conditions:

- Enable/disable toggle checkbox
- Hour slider (0-23)
- Minute slider (0-59)
- Date picker
- When enabled, all intensity calculations recalculate for the simulated time, and a blue info bar shows the simulated parameters

10.5 Tab 1: City Map & Intensity

This is the primary operational view split into two columns.

10.5.1 Left Column - Interactive GIS Map (Folium)

- **Map Tiles:** Dark-themed CartoDB Dark Matter
- **Zone Boundary:** A translucent blue rectangle showing the city jurisdiction
- **Street Light Nodes:** Circular markers on the map, color-coded by intensity:
 - Green (`#4CAF50`): Low intensity, optimized (<40%)
 - Orange (`#ff9800`): Medium intensity (40-70%)

- Red (`#f44336`): High intensity (>70%)
- Purple (`#9c27b0`): Manual override active
- **Popups:** Clicking a node shows: location name, light ID, required intensity, and any active override

10.5.2 Right Column - Intensity Card Grid

- Two-column grid of glass cards, one per street light
- Each card displays:
 - Node ID badge (e.g., "NODE-TVM-A01") in a cyan pill
 - Large intensity percentage in gradient cyan text (e.g., "55%")
 - Location name below
 - Purple "OVERRIDE" badge if manually overridden

10.6 Tab 2: Camera Surveillance

Real-time YOLOv8-powered traffic monitoring split into live feed and analysis.

10.6.1 Controls

- Node selector dropdown to choose which street light the camera monitors
- Start/Stop buttons for the live feed

10.6.2 Live Feed View (Two Columns)

- **Left:** Live camera frame with YOLOv8 bounding boxes drawn around detected vehicles and pedestrians. Caption shows "LIVE - NODE-TVM-A01"
- **Right:** Glass result card showing:
 - Required Intensity in large cyan text
 - TRS (Traffic Risk Score) pill badge
 - Boost percentage pill badge
 - Breakdown: "Base: 55% + Camera: +20%"
 - Detected objects list with individual weights

10.6.3 Below the Feed

- **Manual Override Panel:** Radio buttons (AUTO, MANUAL, OFF) + intensity slider + Apply button
- **24-Hour Forecast Chart:** Plotly line chart showing predicted intensity for the next 24 hours. Cyan line with translucent fill on dark background

10.7 Tab 3: Image Analysis (Light Pollution Profiler)

A diagnostic tool for analyzing uploaded photographs.

10.7.1 Upload and Analysis

- File uploader accepting JPG, JPEG, PNG images

10.7.2 Results Display (Two Columns)

- **Left:**
 - Original uploaded image
 - Brightness heatmap (OpenCV JET colormap applied to grayscale conversion)
- **Right:** Glass result card with:
 - Detected light intensity percentage in large cyan text
 - Danger level badge with color coding:
 - "CRITICAL" (Red): >75% intensity or >50% bright pixels
 - "HIGH" (Orange): >55% intensity or >30% bright pixels
 - "MODERATE" (Yellow): >35% intensity or >15% bright pixels
 - "LOW" (Green): Below all thresholds
 - Assessment description text
 - Metrics grid: Average Brightness, Max Brightness, Brightness StdDev, Bright Pixel %
 - Suggested Optimal Intensity with delta indicator

10.8 Tab 4: Analytics

Energy and environmental impact dashboards.

10.8.1 Top Metrics Row (4 Columns)

- **Avg Required Intensity:** Current average across all nodes
- **Current Draw:** Power consumption in kW (formula: $\text{nodes} * (\text{avg_intensity}/100) * 0.12$)
- **Energy Saved vs 100%:** Savings compared to full-brightness operation
- **CO2 Saved (12h est):** Carbon offset in kg (formula: $\text{saved_kW} * 12\text{h} * 0.82 \text{ kg/kWh}$)

10.8.2 AI Model Performance Table

- Displays model training metrics from `model_metrics`` table:
 - Trained At, Model Type, MAE, RMSE, R-squared, Custom Accuracy %

10.9 Tab 5: Data & Logs

Administrative data views for operators and auditors.

10.9.1 Two-Column Layout

- **Left - Civic Calendar:** Table of celebration events with date, name, intensity multiplier, and city code
- **Right - VIIRS Satellite Data:** Raw radiance readings per node

10.9.2 Add Event Form

- Text field for event name
- Date picker
- Intensity multiplier slider (1.0x to 2.0x)
- City selector (specific city or "ALL")

- Notes/area field
- Submit button that inserts into `celebration\_calendar` table

10.9.3 System Logs

- Latest 50 entries from `system\_log` table showing:
 - Timestamp, Log Level, Component Module, Message
 - Includes DataRefreshWorker entries for weather and prediction refreshes

10.9.4 Recent Camera Detections

- Latest 20 entries from `detection\_events` table showing:
 - Timestamp, Node ID, TRS, Boost %, Final Intensity %

Part 11: Algorithms

11.1 Overall Project Algorithm (Step-by-Step)

- Step 1: Initialize SQLite database with 10 tables.
- Step 2: Register 70 street light nodes across 8 Kerala cities.
- Step 3: Define city zone boundaries (lat/lon bounding boxes).
- Step 4: Populate celebration calendar with Kerala festivals.
- Step 5: Fetch 14 days of historical weather data from Open-Meteo API.
- Step 6: Fetch VIIRS VNP46A1 satellite night-time radiance from NASA LAADS DAAC.
- Step 7: Store weather data and VIIRS radiance into database tables.
- Step 8: For each light node, for each day, for each hour, compute traffic proxy.
- Step 9: Build feature vector X with 10 features per sample.
- Step 10: Generate target intensity y using safety rules and noise injection.
- Step 11: Split dataset into 80% training and 20% testing.
- Step 12: Normalize features using StandardScaler.
- Step 13: Train GradientBoostingRegressor on scaled training data.
- Step 14: Evaluate model on test set (MAE, RMSE, R-squared, Accuracy).
- Step 15: Serialize trained model and scaler to .pkl files.
- Step 16: Store model performance metrics in database.
- Step 17: Generate 24-hour predictions for all 70 nodes (1,680 predictions).
- Step 18: Store predictions in database.
- Step 19: Launch Streamlit dashboard application.
- Step 20: Start DataRefreshWorker background thread (weather 5h, predictions 24h).
- Step 21: Authenticate user via login screen with SHA-256 hashed credentials.
- Step 22: Load street light data and predictions from database for selected city.
- Step 23: Compute real-time intensity using time-curve interpolation.
- Step 24: Apply weather safety override if visibility is poor.
- Step 25: Blend AI prediction with time-curve (70% AI + 30% curve).
- Step 26: Apply celebration multiplier if festival is active.
- Step 27: Display interactive city map with color-coded light nodes.
- Step 28: Display intensity cards for each street light.
- Step 29: Start CameraWorker background thread for live surveillance.
- Step 30: Run YOLOv8n object detection on camera frames.
- Step 31: Compute Traffic Risk Score from detected objects.
- Step 32: Add camera boost to base intensity.
- Step 33: Allow manual override of light intensity by operator.
- Step 34: Display 24-hour forecast chart.
- Step 35: Accept uploaded images for light pollution analysis.
- Step 36: Compute brightness metrics and danger level classification.
- Step 37: Display energy savings and CO2 offset analytics.
- Step 38: Log all system events, detections, and overrides to database.
- Step 39: DataRefreshWorker automatically refreshes weather every 5 hours.

Step 40: DataRefreshWorker automatically regenerates predictions every 24 hours.

11.2 Overall Project Algorithm (With Descriptions)

Step 1: Initialize SQLite database with 10 tables.

The system creates a SQLite database (kerala_smart_lights.db) containing tables for street lights, predictions, weather data, VIIRS satellite data, model metrics, celebration calendar, city zones, detection events, override events, and system logs.

Step 2: Register 70 street light nodes across 8 Kerala cities.

Each node has a unique ID (e.g., NODE-TVM-A01), GPS coordinates (latitude, longitude), location name, wattage, pole height, and installation date. Cities include Thiruvananthapuram, Kochi, Kozhikode, Thrissur, Kollam, Kannur, Alappuzha, Palakkad.

Step 3: Define city zone boundaries (lat/lon bounding boxes).

Each city gets a zone entry with center coordinates and bounding box (north_lat, south_lat, east_lon, west_lon) used for map rendering.

Step 4: Populate celebration calendar with Kerala festivals.

Events like Onam, Vishu, Christmas, and local celebrations are stored with intensity multipliers (1.0x to 2.0x) that boost street light brightness during festivals.

Step 5: Fetch 14 days of historical weather data from Open-Meteo API.

The system calls the Open-Meteo Archive API with coordinates of the reference street light node. It retrieves hourly cloud cover, visibility (meters), and weather codes for the past 14 days.

Step 6: Fetch VIIRS VNP46A1 satellite night-time radiance from NASA LAADS DAAC.

The system queries NASA LAADS DAAC for VIIRS VNP46A1 satellite granules covering each street light's coordinates. HDF5 files are downloaded, and the DNB_At_Sensor_Radiance_500m dataset is extracted to get the regional median radiance value (nW/cm²/sr).

Step 7: Store weather data and VIIRS radiance into database tables.

Weather data goes into the weather_data table with columns for city_code, timestamp, cloudcover, visibility, and is_poor_visibility flag. VIIRS data goes into the viirs_data table with raw_radiance and skyglow_index.

Step 8: For each light node, for each day, for each hour, compute traffic proxy.

Traffic proxy is calculated as: radiance * TrafficFactor(hour), where TrafficFactor uses a sinusoidal function modeling morning and evening rush hour patterns.

Step 9: Build feature vector X with 10 features per sample.

Each training sample has 10 features: hour, weekday, is_weekend, month, is_poor_visibility, raw_radiance, celebration_multiplier, traffic_proxy,

latitude, longitude.

Step 10: Generate target intensity y using safety rules and noise injection.

Daytime (6 AM - 6 PM): $y = 0$ unless poor visibility, then $y = \text{MIN}(100, 50 + \text{proxy})$.

Nighttime: $y = \text{MIN}(100, (25 + \text{proxy} * 5) * \text{celebration_mult})$.

Safety floor: $y = \text{MAX}(y, 40)$ normally, $y = \text{MAX}(y, 55)$ in poor visibility.

Gaussian noise $\text{Normal}(0, 6)$ is added to prevent model overfitting.

Step 11: Split dataset into 80% training and 20% testing.

scikit-learn `train_test_split` divides the data with a fixed random seed (42) for reproducibility.

Step 12: Normalize features using StandardScaler.

StandardScaler fits on training data and transforms both training and test sets to zero mean and unit variance, ensuring all features contribute equally.

Step 13: Train GradientBoostingRegressor on scaled training data.

The model uses 200 boosting stages (decision trees), learning rate of 0.1, maximum tree depth of 4, and minimizes squared error loss. Each tree corrects the residual errors of the previous ensemble.

Step 14: Evaluate model on test set (MAE, RMSE, R-squared, Accuracy).

MAE measures average absolute error. RMSE penalizes large errors more heavily.

R-squared indicates variance explained (target ~0.92). Custom accuracy counts predictions within 5% of actual values.

Step 15: Serialize trained model and scaler to .pkl files.

Python pickle serializes the trained GradientBoostingRegressor to `intensity_model.pkl` and the fitted StandardScaler to `intensity_scaler.pkl` for loading by the dashboard application.

Step 16: Store model performance metrics in database.

MAE, RMSE, R-squared, custom accuracy, model type, and training timestamp are stored in the `model_metrics` table for display in the Analytics tab.

Step 17: Generate 24-hour predictions for all 70 nodes (1,680 predictions).

For each of the 70 street lights and each of the 24 hours, the trained model predicts the required intensity. Each prediction includes confidence bounds (predicted +/- 10%), day type, celebration multiplier, and weather override flag.

Step 18: Store predictions in database.

All 1,680 prediction rows are inserted into the predictions table with `prediction_made_at` timestamp, `target_timestamp`, `light_id`, and predicted values.

Step 19: Launch Streamlit dashboard application.

The Streamlit app (`app.py`) starts on `localhost:8501` with a wide layout, glassmorphism dark theme, and 5 interactive tabs.

Step 20: Start DataRefreshWorker background thread (weather 5h, predictions 24h).

A singleton daemon thread is created via `st.cache_resource`. It checks every 60 seconds whether weather data (5-hour cycle) or predictions (24-hour cycle) need refreshing, and automatically fetches fresh data from APIs.

Step 21: Authenticate user via login screen with SHA-256 hashed credentials.

Users select their municipality, enter username and password. The password is SHA-256 hashed and compared against `credentials.json`. Admin users can switch between cities after login.

Step 22: Load street light data and predictions from database for selected city.

The dashboard queries `street_lights`, `predictions`, `weather_data`, `viirs_data`, and `city_zones` tables filtered by the authenticated user's city code.

Step 23: Compute real-time intensity using time-curve interpolation.

A 21-point anchor curve defines intensity targets across 24 hours. The current time ($\text{hour} + \text{minute}/60$) is linearly interpolated between the two nearest anchor points to produce a smooth, continuous intensity value.

Step 24: Apply weather safety override if visibility is poor.

If the latest weather data shows visibility below 3000 meters, the intensity is raised to at least 60%, even during daytime. This ensures road safety during monsoon storms, fog, or heavy rain.

Step 25: Blend AI prediction with time-curve (70% AI + 30% curve).

During nighttime hours, the ML model's prediction is blended with the time-curve interpolation: $\text{final} = 0.7 * \text{AI_prediction} + 0.3 * \text{curve_value}$. This prevents abrupt jumps while leveraging AI accuracy.

Step 26: Apply celebration multiplier if festival is active.

If today's date matches a `celebration_calendar` entry for the city (or "ALL"), the intensity is multiplied by the event multiplier (e.g., 1.5x for festivals) and capped at 100%.

Step 27: Display interactive city map with color-coded light nodes.

Folium renders a CartoDB Dark Matter map with the city zone boundary rectangle and circle markers for each light. Colors: green (<40%), orange (40-70%), red (>70%), purple (manual override active).

Step 28: Display intensity cards for each street light.

A two-column grid of glass-styled cards shows each node's ID, location name, and required intensity percentage in large cyan text.

Step 29: Start CameraWorker background thread for live surveillance.

When the user clicks "Start Live Feed", a daemon thread opens the webcam via OpenCV DirectShow backend, captures frames at native resolution, and runs YOLOv8n inference every 4th frame.

Step 30: Run YOLOv8n object detection on camera frames.

The YOLOv8 nano model detects objects with confidence threshold 0.40 and IoU threshold 0.45. It identifies persons, bicycles, cars, motorcycles, buses, and trucks, drawing bounding boxes on the frame.

Step 31: Compute Traffic Risk Score from detected objects.

Each detected object contributes a weighted score: Person=1.0, Bicycle=1.2, Car=2.0, Motorcycle=1.5, Bus=3.5, Truck=3.5. The TRS is the sum of all detected object weights.

Step 32: Add camera boost to base intensity.

TRS is converted to an intensity boost: $TRS \geq 15$ gives +40%, $TRS \geq 10$ gives +30%, $TRS \geq 5$ gives +20%, $TRS \geq 2$ gives +10%. Final intensity = $\text{MIN}(100, \text{base} + \text{boost})$.

Step 33: Allow manual override of light intensity by operator.

Operators can set any light to MANUAL mode (custom intensity), OFF mode (0%), or AUTO mode (revert to AI). Overrides are logged to `override_events` table and shown as purple markers on the map.

Step 34: Display 24-hour forecast chart.

A Plotly line chart shows predicted intensity for the next 24 hours for the selected light node, with cyan line, translucent fill, on dark background.

Step 35: Accept uploaded images for light pollution analysis.

Users upload JPG/PNG images in the Image Analysis tab. The system decodes the image using OpenCV and converts it to grayscale for brightness analysis.

Step 36: Compute brightness metrics and danger level classification.

Average brightness, max brightness, standard deviation, and bright pixel percentage are calculated. Danger levels: CRITICAL (>75% or >50% bright pixels), HIGH (>55%), MODERATE (>35%), LOW (below all thresholds). A JET colormap heatmap is generated for visualization.

Step 37: Display energy savings and CO2 offset analytics.

Current power draw = $\text{nodes} * (\text{avg_intensity}/100) * 0.12 \text{ kW}$.

Energy saved = $\text{nodes} * ((100 - \text{avg_intensity})/100) * 0.12 \text{ kW}$.

CO2 saved = $\text{saved_kW} * 12 \text{ hours} * 0.82 \text{ kg/kWh emission factor}$.

Step 38: Log all system events, detections, and overrides to database.

System logs capture DataRefreshWorker operations, camera detections are stored in `detection_events` (throttled to every 5 seconds), and manual overrides are recorded in `override_events` with timestamps, user, and previous/new values.

Step 39: DataRefreshWorker automatically refreshes weather every 5 hours.

Every 60 seconds, the worker checks if weather data is older than 5 hours.

If stale, it fetches fresh data from Open-Meteo and updates the `weather_data` table for all city codes.

Step 40: DataRefreshWorker automatically regenerates predictions every 24 hours.

Every 60 seconds, the worker checks if predictions are older than 24 hours. If stale, it loads the pickled model, fetches fresh weather and VIIRS data, regenerates all 1,680 predictions, and replaces the predictions table.

11.3 Overall Project Pseudo Algorithm

```
ALGORITHM: LumiCity_Complete_System
=====

// =====
// PHASE 1: DATABASE INITIALIZATION
// =====

PROCEDURE InitializeSystem():
    CREATE DATABASE "kerala_smart_lights.db"
    CREATE TABLES: street_lights, predictions, weather_data,
        viirs_data, model_metrics, celebration_calendar,
        city_zones, detection_events, override_events, system_log
    INSERT 70 street light nodes across 8 Kerala cities
    INSERT city zone boundaries for each city
    INSERT celebration calendar entries (Kerala festivals)

// =====
// PHASE 2: DATA COLLECTION
// =====

PROCEDURE CollectExternalData():
    FOR each day D in [today - 14 ... today]:
        lat, lon = reference_light_node.coordinates
        weather[D] = HTTP_GET(Open-Meteo API, lat, lon, D)
        STORE weather[D] INTO weather_data TABLE

        FOR each light L in street_lights:
            radiance[L] = HTTP_GET(NASA LAADS DAAC, L.lat, L.lon)
            IF radiance[L] is NULL:
                radiance[L] = 15.4 // Regional fallback
            STORE radiance[L] INTO viirs_data TABLE

// =====
// PHASE 3: FEATURE ENGINEERING
// =====

PROCEDURE BuildTrainingData():
    X = [] // Feature matrix
    Y = [] // Target vector

    FOR each day D in 14-day range:
        FOR each light L in street_lights:
            FOR each hour H in [0 ... 23]:
                traffic_proxy = radiance[L] * TrafficFactor(H)
                is_weekend = (D.weekday >= 5)
                is_poor_vis = (weather[D][H].visibility < 3000)
                cel_mult = LookupCelebration(D, L.city_code)

                // Compute ideal target intensity
                IF H >= 6 AND H < 18: // Daytime
                    IF is_poor_vis:
                        y = MIN(100, 50 + traffic_proxy)
                    ELSE:
                        y = 0 // Lights OFF
                ELSE: // Nighttime
                    y = MIN(100, (25 + traffic_proxy * 5) * cel_mult)
                IF is_poor_vis:
```

```

        y = MAX(y, 55)
    ELSE:
        y = MAX(y, 40) // Safety floor

    y = y + GaussianNoise(mean=0, std=6)
    y = CLIP(y, 0, 100)

    X.append([H, D.weekday, is_weekend, D.month,
              is_poor_vis, radiance[L], cel_mult,
              traffic_proxy, L.lat, L.lon])
    Y.append(y)

RETURN X, Y

// =====
// PHASE 4: MODEL TRAINING
// =====

PROCEDURE TrainModel(X, Y):
    X_train, X_test, y_train, y_test = SPLIT(X, Y, ratio=0.8)
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)

    model = GradientBoostingRegressor(
        n_estimators=200,
        learning_rate=0.1,
        max_depth=4,
        random_state=42
    )
    model.FIT(X_train, y_train)

    y_pred = model.PREDICT(X_test)
    MAE = MEAN(ABS(y_pred - y_test))
    RMSE = SQRT(MEAN((y_pred - y_test)^2))
    R2 = 1 - SUM((y_test - y_pred)^2) / SUM((y_test - MEAN(y_test))^2)

    SERIALIZE model TO "intensity_model.pkl"
    SERIALIZE scaler TO "intensity_scaler.pkl"
    STORE metrics INTO model_metrics TABLE

// =====
// PHASE 5: PREDICTION GENERATION
// =====

PROCEDURE GeneratePredictions(model, scaler):
    prediction_rows = []
    FOR each light L in street_lights:
        FOR each hour H in [0 ... 23]:
            features = BuildFeatureVector(L, H, today)
            intensity = model.PREDICT(scaler.TRANSFORM(features))
            intensity = CLIP(intensity, 0, 100)
            prediction_rows.APPEND(intensity, L.light_id, H)
    DELETE all FROM predictions TABLE
    INSERT prediction_rows INTO predictions TABLE

// =====
// PHASE 6: DASHBOARD LAUNCH
// =====

PROCEDURE LaunchDashboard():
    START Streamlit application on localhost:8501
    APPLY glassmorphism CSS theme

    // Start background workers
    refresh_worker = DataRefreshWorker(pred_interval=24h, wx_interval=5h)
    refresh_worker.START() // Daemon thread

```

```

// Authentication
DISPLAY login screen
credentials = READ "credentials.json"
IF SHA256(input_password) == credentials[username]:
    SET session authenticated
    SET session city_code
ELSE:
    SHOW error, STOP

// =====
// PHASE 7: REAL-TIME INTENSITY COMPUTATION
// =====

PROCEDURE ComputeIntensity(light_id, hour, minute, date, city):
    // Step A: Time-curve interpolation
    t = hour + minute / 60.0
    base = LinearInterpolate(t, anchor_points_21)

    // Step B: Weather safety override
    IF LatestWeather(city).visibility < 3000:
        base = MAX(base, 60.0)

    // Step C: AI prediction blending
    ai_pred = QueryPrediction(light_id, hour)
    IF hour >= 6 AND hour < 18: // Daytime
        IF poor_visibility:
            base = MAX(base, ai_pred)
    ELSE: // Nighttime
        base = 0.7 * ai_pred + 0.3 * base

    // Step D: Celebration multiplier
    mult = QueryCelebration(date, city)
    IF mult > 1.0:
        base = MIN(100, base * mult)

    // Step E: Camera traffic boost
    IF camera_active:
        trs = ComputeTRS(camera_frame)
        boost = TRS_TO_BOOST(trs)
        base = MIN(100, base + boost)

    RETURN ROUND(base, 1)

// =====
// PHASE 8: CAMERA SURVEILLANCE
// =====

PROCEDURE CameraSurveillance():
    OPEN webcam via OpenCV
    LOAD YOLOv8n model

    LOOP every frame:
        frame = CAPTURE from webcam
        IF frame_count MOD 4 == 0:
            detections = YOLO.DETECT(frame, conf=0.40)
            trs = 0.0
            FOR each detection:
                IF class IN {Person,Bicycle,Car,Motorcycle,Bus,Truck}:
                    trs = trs + COCO_WEIGHT[class]
                DRAW bounding box on frame

            boost = MAP_TRS_TO_BOOST(trs)
            final = MIN(100, base_intensity + boost)
            STORE detection event TO database (every 5 seconds)

// =====
// PHASE 9: IMAGE ANALYSIS
// =====

```

```

PROCEDURE AnalyzeImage(uploaded_image):
    image = DECODE(uploaded_image)
    gray = CONVERT_TO_GRAYSCALE(image)
    avg_brightness = MEAN(gray)
    intensity_pct = (avg_brightness / 255) * 100
    bright_pixels = COUNT(gray > 200) / TOTAL_PIXELS * 100

    IF intensity_pct > 75 OR bright_pixels > 50:
        danger = "CRITICAL"
    ELIF intensity_pct > 55 OR bright_pixels > 30:
        danger = "HIGH"
    ELIF intensity_pct > 35 OR bright_pixels > 15:
        danger = "MODERATE"
    ELSE:
        danger = "LOW"

    heatmap = APPLY_COLORMAP(gray, JET)
    DISPLAY results with danger level and suggested intensity

// =====
// PHASE 10: AUTOMATIC DATA REFRESH
// =====

PROCEDURE AutoRefreshLoop(): // Runs as background daemon thread
    WHILE application is running:
        WAIT 60 seconds

        IF predictions older than 24 hours:
            LOAD model.pkl, scaler.pkl
            FETCH fresh weather from Open-Meteo API
            FETCH fresh VIIRS data from NASA
            REGENERATE all 1,680 predictions
            UPDATE predictions table
            LOG "Full refresh complete"

        ELIF weather older than 5 hours:
            FETCH fresh weather from Open-Meteo API
            UPDATE weather_data table
            LOG "Weather refreshed (5h cycle)"

// =====
// MAIN EXECUTION
// =====

PROCEDURE MAIN():
    InitializeSystem()
    CollectExternalData()
    X, Y = BuildTrainingData()
    TrainModel(X, Y)
    GeneratePredictions(model, scaler)
    LaunchDashboard()
    // Dashboard runs continuously with:
    //   - Real-time intensity computation (Phase 7)
    //   - Camera surveillance (Phase 8)
    //   - Image analysis on demand (Phase 9)
    //   - Auto refresh in background (Phase 10)

```

11.4 Component Algorithm 1: Training Data Generation

ALGORITHM: GenerateTrainingData
 INPUT: street_lights table, 14-day date range
 OUTPUT: Feature matrix X, Target vector y

1. FOR each day D in [today - 14 ... today]:

```

2.     Fetch weather data: W = FetchWeather(lat, lon, D)
3.     Fetch VIIRS radiance: R = FetchVIIRS(lat, lon, D)
4.     FOR each street light node L:
5.         FOR each hour H in [0 ... 23]:
6.             Compute traffic_proxy = R * TrafficFactor(H)
7.             Determine is_weekend = (D.weekday >= 5)
8.             Look up celebration_multiplier from calendar
9.             Set is_poor_visibility = (W.visibility < 3000)
10.
11.             IF H >= 6 AND H < 18:
12.                 IF is_poor_visibility:
13.                     y = MIN(100, 50 + traffic_proxy)
14.                 ELSE:
15.                     y = 0
16.             ELSE:
17.                 y = MIN(100, (25 + traffic_proxy * 5) * celebration_mult)
18.                 IF is_poor_visibility:
19.                     y = MAX(y, 55)
20.                 ELSE:
21.                     y = MAX(y, 40)
22.
23.             y = y + Normal(mean=0, std=6)
24.             y = CLIP(y, 0, 100)
25.
26.             X.append([H, D.weekday, is_weekend, D.month,
27.                     is_poor_visibility, R, celebration_mult,
28.                     traffic_proxy, L.lat, L.lon])
29.             Y.append(y)
30.
31. RETURN X, Y

```

11.5 Component Algorithm 2: Model Training Pipeline

ALGORITHM: TrainIntensityModel

INPUT: Feature matrix X, Target vector y

OUTPUT: Trained model (intensity_model.pkl), Scaler (intensity_scaler.pkl)

```

1. Split data: X_train, X_test, y_train, y_test = split(X, y, ratio=80/20)
2. Initialize StandardScaler
3. X_train_scaled = Scaler.fit_transform(X_train)
4. X_test_scaled = Scaler.transform(X_test)
5.
6. Initialize GradientBoostingRegressor:
7.     n_estimators = 200
8.     learning_rate = 0.1
9.     max_depth = 4
10.    random_state = 42
11.
12. model.fit(X_train_scaled, y_train)
13.
14. predictions = model.predict(X_test_scaled)
15. MAE = mean(|predictions - y_test|)
16. RMSE = sqrt(mean((predictions - y_test)^2))
17. R2 = 1 - sum((y_test - predictions)^2) / sum((y_test - mean(y_test))^2)
18. Accuracy = count(|predictions - y_test| <= 5) / len(y_test) * 100
19.
20. Serialize model to intensity_model.pkl
21. Serialize scaler to intensity_scaler.pkl
22. Store metrics in model_metrics table
23.
24. RETURN model, scaler

```

11.6 Component Algorithm 3: Real-Time Intensity Computation

ALGORITHM: ComputeBaseIntensity

INPUT: light_id, hour, minute, date, city_code

OUTPUT: Required intensity percentage (0-100)

```
1. t = hour + minute / 60.0
2. anchors = [(0.0, 40), (0.5, 32), (1.0, 20), (3.0, 18),
3.           (5.0, 20), (5.5, 10), (6.0, 0), (17.5, 0),
4.           (18.0, 40), (18.25, 48), (18.5, 55), (19.0, 70),
5.           (19.5, 69), (20.0, 66), (20.5, 62), (21.0, 55),
6.           (21.5, 50), (22.0, 48), (22.5, 44), (23.0, 40),
7.           (23.5, 38), (24.0, 40)]
8.
9. Find anchors[i], anchors[i+1] where anchors[i].time <= t
10. fraction = (t - t0) / (t1 - t0)
11. base = v0 + (v1 - v0) * fraction
12.
13. Query latest weather_data for city_code
14. IF is_poor_visibility:
15.     base = MAX(base, 60.0)
16.
17. Query predictions table for light_id at target hour
18. IF prediction exists AND predicted_intensity > 0:
19.     IF hour >= 6 AND hour < 18:
20.         IF is_poor_visibility:
21.             base = MAX(base, ai_prediction)
22.     ELSE:
23.         base = 0.7 * ai_prediction + 0.3 * base
24.
25. Query celebration_calendar for date and city
26. IF event exists:
27.     base = MIN(100, base * max_multiplier)
28.
29. RETURN round(base, 1)
```

11.7 Component Algorithm 4: Traffic Risk Score (TRS)

ALGORITHM: ComputeTRS

INPUT: Camera frame (BGR image)

OUTPUT: Traffic Risk Score, Intensity Boost percentage

```
1. Pass frame through YOLOv8n model (confidence=0.40, IoU=0.45)
2. Initialize trs = 0.0
3.
4. FOR each detected bounding box:
5.     class_id = box.class
6.     IF class_id IN {Person, Bicycle, Car, Motorcycle, Bus, Truck}:
7.         weight = COCO_WEIGHTS[class_id]
8.         trs = trs + weight
9.         Draw bounding box and label on frame
10.
11. IF trs >= 15.0:    boost = 40%
12. ELIF trs >= 10.0: boost = 30%
13. ELIF trs >= 5.0:  boost = 20%
14. ELIF trs >= 2.0:  boost = 10%
15. ELSE:             boost = 0%
16.
17. final_intensity = MIN(100, base_intensity + boost)
18.
19. RETURN trs, boost, final_intensity
```

11.8 Component Algorithm 5: Traffic Factor Proxy

ALGORITHM: GetTrafficFactor

INPUT: hour (0-23)

OUTPUT: Traffic load factor (0.1 to ~1.5)

```
1. primary   = SIN(pi * (hour - 4) / 12)
2. secondary = 0.5 * COS(pi * hour / 6)
3. factor    = primary + secondary
4. RETURN MAX(0.1, factor)
```

11.9 Component Algorithm 6: Automatic Data Refresh

ALGORITHM: DataRefreshWorker

INPUT: None (runs as background daemon thread)

OUTPUT: Updated weather_data and predictions in database

```
1. ON STARTUP:
2.   Read last prediction timestamp from database
3.   Read last weather fetch timestamp from database
4.   IF predictions are stale (>24 hours old):
5.     Execute FullRefresh()
6.   ELIF weather is stale (>5 hours old):
7.     Execute WeatherRefresh()
8.
9. MAIN LOOP (every 60 seconds):
10.  IF stop_event is set: EXIT
11.  IF predictions are stale (>24 hours old):
12.    Execute FullRefresh()
13.  ELIF weather is stale (>5 hours old):
14.    Execute WeatherRefresh()
15.
16. PROCEDURE WeatherRefresh():
17.   Fetch fresh weather from Open-Meteo API
18.   Update weather_data table for all city codes
19.   Log to system_log
20.
21. PROCEDURE FullRefresh():
22.   Load intensity_model.pkl and intensity_scaler.pkl
23.   Fetch weather from Open-Meteo API
24.   Update weather_data table
25.   FOR each street light node:
26.     Fetch VIIRS radiance for node coordinates
27.     Look up celebration multiplier
28.     FOR each hour H in [0 ... 23]:
29.       Build feature vector
30.       predicted_intensity = model.predict(scaler.transform(features))
31.       Clip to [0, 100]
32.       Append to prediction_rows
33.   Replace all predictions in database
34.   Log to system_log
```

11.10 Component Algorithm 7: Image Light Pollution Analysis

ALGORITHM: AnalyzeImagePollution

INPUT: Uploaded image (JPG/PNG)

OUTPUT: Danger level, metrics, suggested intensity

```
1. Decode image from byte buffer using OpenCV
2. Convert to grayscale: gray = BGR_to_GRAY(image)
3.
4. avg_brightness = MEAN(gray)
5. max_brightness = MAX(gray)
6. brightness_std = STD(gray)
7. intensity_pct = (avg_brightness / 255) * 100
8. bright_pixels = COUNT(gray > 200) / total_pixels * 100
9.
10. IF intensity_pct > 75 OR bright_pixels > 50:
```

```

11.     danger = "CRITICAL"
12. ELIF intensity_pct > 55 OR bright_pixels > 30:
13.     danger = "HIGH"
14. ELIF intensity_pct > 35 OR bright_pixels > 15:
15.     danger = "MODERATE"
16. ELSE:
17.     danger = "LOW"
18.
19. heatmap = ApplyColorMap(gray, JET_COLORMAP)
20.
21. IF intensity_pct > 60:
22.     suggested = MAX(20, intensity_pct * 0.5)
23. ELIF intensity_pct > 40:
24.     suggested = intensity_pct * 0.7
25. ELSE:
26.     suggested = intensity_pct
27.
28. RETURN danger, intensity_pct, bright_pixels, suggested, heatmap

```

11.11 Component Algorithm 8: Skyglow Ecological Index

ALGORITHM: ComputeSkyglow

INPUT: light_intensities (dict), city_code

OUTPUT: Skyglow index value, color classification

```

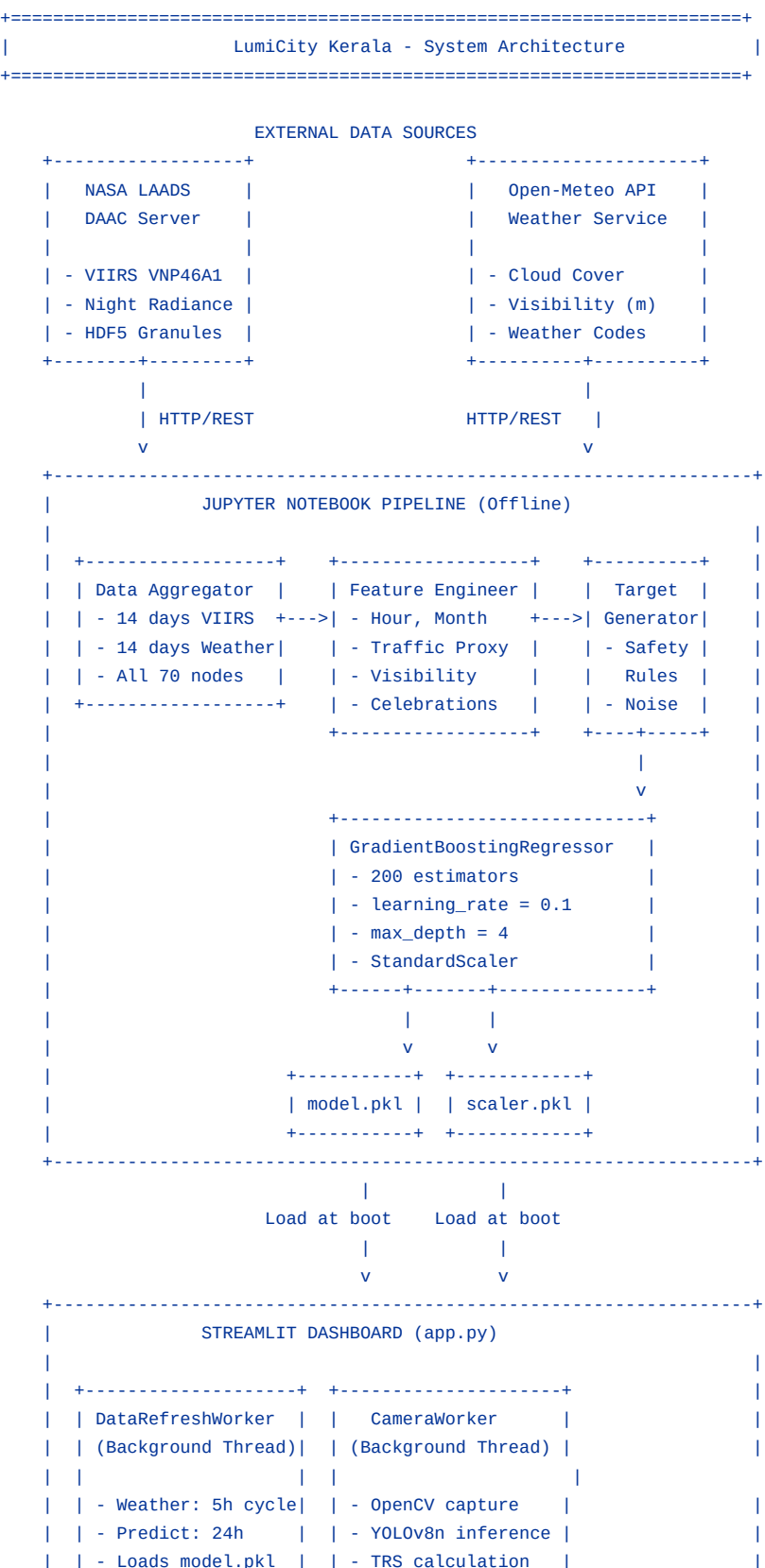
1. avg_intensity = MEAN(all light_intensities values)
2. avg_radiance = Query AVG(raw_radiance) from viirs_data for city
3. IF no radiance data available:
4.     avg_radiance = 5.0 (default)
5.
6. skyglow = avg_radiance * (avg_intensity / 100.0)
7.
8. IF skyglow > 1.5: color = RED
9. ELIF skyglow > 0.8: color = ORANGE
10. ELSE: color = GREEN
11.
12. RETURN skyglow, color

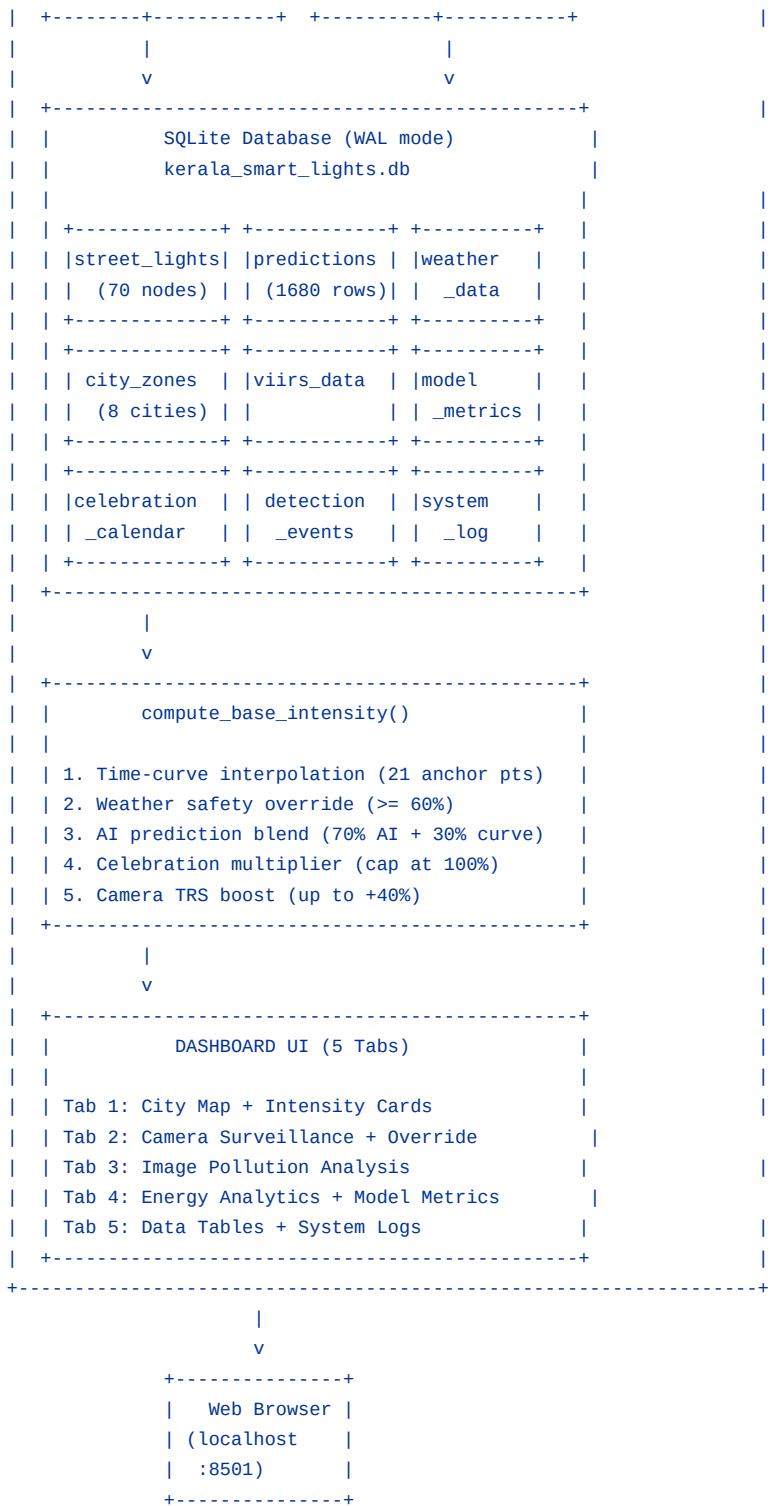
```

Part 12: System Block Diagram

12.1 Complete System Block Diagram

The following ASCII block diagram illustrates the entire LumiCity system architecture in a simple, easy-to-understand format.





12.2 Data Flow Diagram



```

Open-Meteo ----[JSON]-----> fetch_real_weather()
                                |
                                v
                                weather_data table
                                (cloudcover, visibility)

```

Phase 2: Model Training (Jupyter Notebook - One-time)

```
=====
```

```

viirs_data + weather_data + street_lights + celebration_calendar
    |           |           |           |
    v           v           v           v
+-----+-----+
|           Feature Matrix X (10 columns)           |
| [hour, weekday, is_weekend, month,                  |
| is_poor_vis, radiance, celebration_mult,            |
| traffic_proxy, latitude, longitude]                |
+-----+-----+
    |                               |
    v                               v
X_train (80%)                     X_test (20%)
    |                               |
    v                               v
StandardScaler.fit_transform      StandardScaler.transform
    |                               |
    v                               v
GBR.fit(X_train, y_train)          GBR.predict(X_test)
    |                               |
    v                               v
intensity_model.pkl               Evaluation Metrics
intensity_scaler.pkl              (MAE, RMSE, R2)

```

Phase 3: Prediction Generation (Auto-Refresh / 24h)

```
=====
```

```

model.pkl + scaler.pkl
    |
    v
FOR each node (70 lights):
    FOR each hour (0-23):
        features --> scaler.transform --> model.predict
                                |
                                v
                                predictions table
                                (1680 rows: 70 x 24)

```

Phase 4: Real-Time Dashboard (Continuous)

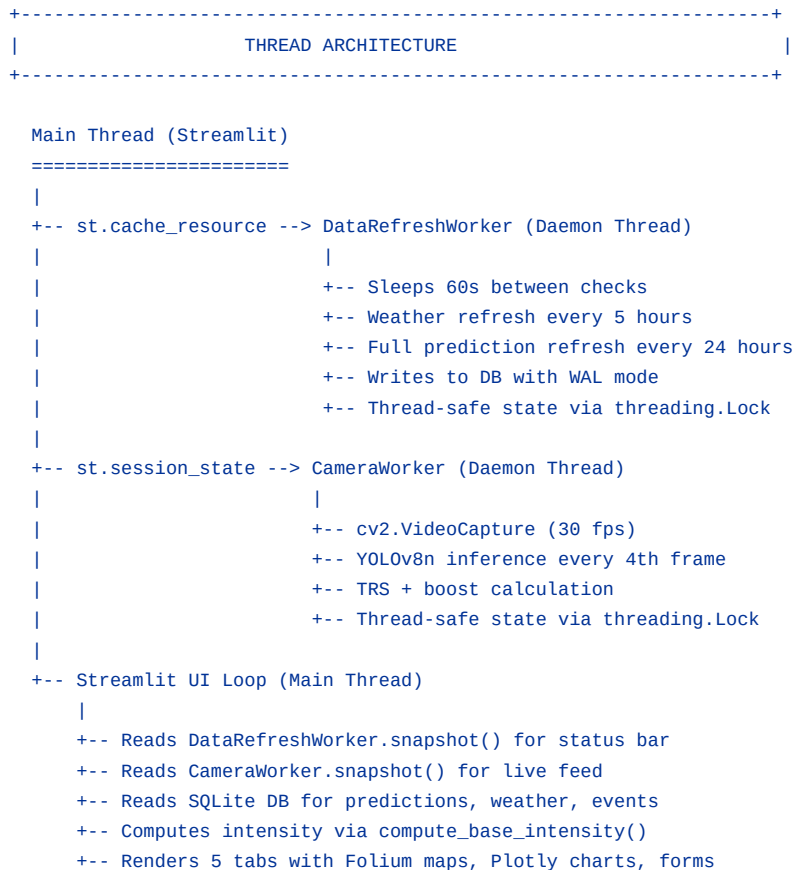
```
=====
```

```

predictions + weather_data + celebration_calendar + time_curve
    |           |           |           |
    +-----+-----+-----+-----+           |
    |           |           |           |           |
    v           v           |           v           |
compute_base_intensity(light_id, hour, minute)
    |
    +--- Camera TRS boost (if active) ---+
    |           |           |           |           |
    v           v           |           v           |
final_intensity = MIN(100, base + boost)
    |
    v
Dashboard UI renders map + cards + charts

```

12.3 Thread Architecture Diagram



12.4 Authentication Flow



Part 13: Automatic Data Refresh System

13.1 Problem Statement

In the original LumiCity architecture, data collection (weather fetching, VIIRS satellite radiance extraction) and prediction generation only occurred when the Jupyter notebook cells were manually re-executed. After 24 hours, predictions became stale and the dashboard was operating on outdated data. Weather conditions could change dramatically within hours, making the static predictions inaccurate.

13.2 Solution: DataRefreshWorker

The `DataRefreshWorker` is a background daemon thread that automatically refreshes data on two independent cycles:

- **Weather Refresh (Every 5 Hours):** Fetches fresh weather data from the Open-Meteo API and updates the `weather_data` table. This frequent cycle ensures that visibility overrides are based on current atmospheric conditions.
- **Full Prediction Refresh (Every 24 Hours):** Fetches weather + VIIRS satellite data, loads the pickled ML model, and regenerates all 1,680 predictions (70 lights x 24 hours). This replaces the entire `predictions` table with fresh data.

13.3 Architecture Design

The worker follows the same daemon-thread pattern as the existing `CameraWorker`:

```
DataRefreshWorker
|
+-- __init__(interval=86400, weather_interval=18000)
|   +-- Initialize state dict: last_refresh, last_weather, status, error
|   +-- _init_last_refresh_from_db() // Read timestamps from DB at startup
|
+-- start() // Launch background thread
+-- stop() // Signal thread to exit
+-- snapshot() // Thread-safe state read (for UI status bar)
+-- is_running() // Check thread health
|
+-- _run() // Main thread loop
    +-- On startup: check if data is stale, refresh if needed
    +-- Every 60 seconds: check both cycles
        +-- _predictions_are_stale() --> _do_full_refresh()
        +-- _weather_is_stale() --> _do_weather_refresh()
```

13.3.1 Singleton Pattern

The worker is instantiated exactly once using Streamlit's `@st.cache_resource` decorator:

```
@st.cache_resource
def get_data_refresh_worker():
    worker = DataRefreshWorker(interval=REFRESH_INTERVAL_SECONDS)
    worker.start()
```

```
return worker
```

```
_refresh_worker = get_data_refresh_worker()
```

This ensures that no matter how many times Streamlit reruns the script (on user interactions), only one worker thread exists.

13.4 Weather Refresh Cycle (5 Hours)

The ``_do_weather_refresh()`` method:

1. Reads the first street light's coordinates as the reference point
2. Calls ``fetch_real_weather()`` to get today's hourly weather from Open-Meteo
3. Populates the ``weather_data`` table for all city codes:
 - Deletes existing entries for today
 - Inserts fresh rows with cloudcover, visibility, is_poor_visibility flag
4. Updates the worker's ``last_weather`` timestamp
5. Logs the event to ``system_log``

This 5-hour cycle ensures that weather-based safety overrides (poor visibility triggers 60% minimum intensity) reflect current conditions rather than stale data from the initial notebook run.

13.5 Full Prediction Refresh Cycle (24 Hours)

The ``_do_full_refresh()`` method:

1. Loads ``intensity_model.pkl`` and ``intensity_scaler.pkl`` from disk
2. Fetches today's weather data (same as weather cycle)
3. Populates ``weather_data`` table
4. For each of the 70 street light nodes:
 - a. Fetches VIIRS satellite radiance for the node's coordinates
 - b. Looks up celebration multiplier for today's date
 - c. For each of the 24 hours:
 - Determines visibility status
 - Computes traffic proxy
 - Builds the 10-feature vector
 - Runs ``scaler.transform()`` then ``model.predict()``
 - Clips prediction to [0, 100]
 - Appends prediction row
5. Deletes all existing predictions from the database
6. Inserts all 1,680 new prediction rows
7. Updates both ``last_refresh`` and ``last_weather`` timestamps
8. Logs the event to ``system_log``

13.6 SQLite Concurrency with WAL Mode

Since the background worker writes to the database while the UI thread reads from it, all database operations use:

```
conn = sqlite3.connect(DB_NAME, timeout=10)
conn.execute("PRAGMA journal_mode=WAL")
```

Write-Ahead Logging (WAL) mode allows concurrent readers and a single writer without blocking. The `timeout=10` parameter ensures operations wait up to 10 seconds for a lock rather than failing immediately.

13.7 Status Dashboard Integration

The worker's state is displayed in the status bar through two indicators:

- ****WX Sync:**** Shows the weather refresh cycle status
 - Green dot + time since last weather fetch
 - Cyan dot + "Updating..." when a weather refresh is in progress
 - Orange dot + "Never" if weather has never been fetched
- ****Predict:**** Shows the prediction refresh cycle status
 - Green dot + time since last prediction generation
 - Cyan dot + "Syncing..." when predictions are being generated
 - Orange dot + "Never" if predictions have never been generated
 - Red dot + "Error" if the last refresh attempt failed

13.8 Startup Behavior

On application startup, the worker intelligently checks existing data:

1. Reads the most recent `prediction\_made\_at` timestamp from the `predictions` table
2. Reads the most recent `fetched\_at` timestamp from the `weather\_data` table
3. If predictions exist from the notebook run, the status bar immediately shows the correct timestamp instead of "Never"
4. The worker then checks if either cycle is due and refreshes accordingly

This means the dashboard never shows misleading "Never" status when valid data already exists from the initial notebook execution.

13.9 Error Handling and Recovery

- If a weather API call fails, the worker reverts to its previous status and logs the error
- If the full refresh fails (e.g., model files missing), the error is stored in the worker state and displayed as a red "Error" indicator
- All errors are logged to the `system\_log` table for debugging
- The worker continues running after errors and retries at the next cycle interval
- VIIRS data falls back to a regional median of 15.4 W/cm²/sr if the NASA API is unreachable

